A STEP-BY-STEP TEXTBOOK

Transformers from First Principles

The prerequisites, *Attention Is All You Need*, and the Transformer architecture, explained one idea at a time.

Dot products → Softmax → Neural networks → RNNs → Attention → Encoder & decoder → Training & generation → Modern LLMs

Theory edition · companion to the *Transformer Room* course
October 2026

PREFACE

How to use this book

This book teaches the Transformer, the architecture behind today's language models, from the ground up. It starts with the five pieces of mathematics the model is built from. It then covers just enough neural-network and sequence-model background to make the motivation clear. After that it assembles attention and the full encoder–decoder model of *Attention Is All You Need* (Vaswani et al., 2017), one idea per chapter.

Structure

- **Parts and chapter codes.** Chapters are numbered by part letter: A1–A5 for mathematics, B1–B5 for neural networks, C1–C5 for text and sequences, D1–D8 for attention, E1–E8 for the Transformer itself, and G1–G3 for later developments. A reference such as "(D4)" in the text points to that chapter. The codes match the Transformer Room web course, so you can move between the two.
- **Prerequisites line.** Under every chapter title, the earlier chapters it relies on are listed. If something feels unclear, go back to those chapters first.
- **One running example.** The sentence pair *Ich mag Tee* → *I like tea* and the toy vectors I = [1,0], like = [0,1], tea = [1,1] are used throughout, so each new ingredient can be compared with the last.
- **Worked examples, summaries and exercises.** Every chapter has hand-calculated examples, ends with a short summary, and has exercises marked *Compute* or *Explain*. Solutions are in Appendix B.
- **Review chapters** (B5, C5, D8) mix questions from several chapters. Answers are in Appendix C.
- **Appendices:** A, notation and glossary; B, solutions to exercises; C, answers to the self-assessment and review questions; D, references.

A suggested four-week plan

Week	Read	Goal by the end of the week
1	Part 0, Part A, Part B	compute softmax and a gradient-descent step by hand
2	Part C, then D1–D4	compute scaled dot-product attention for three tokens
3	D5–D8, E1–E4	draw the encoder and decoder blocks from memory
4	E5–E8, Part G, then the paper	read the paper in three passes and explain every box in its Figure 1

Read actively. Work each example yourself before reading its result, and attempt the exercises before checking the solutions. Spaced review matters too: retake the review chapters a week after you first read them.

Contents

How to use this book 2

Part 0 Orientation 5

0.1 The destination: the whole Transformer in one picture 6

0.2 Readiness check: where should you start? 9

Part A Math you'll actually use 11

A1 Vectors and the dot product as similarity 12

A2 Matrices, shapes and transpose 15

A3 Exponentials, logarithms and softmax 18

A4 Derivatives and the chain rule 20

A5 Mean, variance and normalization 23

Part B Neural networks 25

B1 Neurons and layers, starting from familiar ML 26

B2 Measuring error: squared error and cross-entropy 28

B3 Gradient descent and backpropagation 30

B4 Training in practice: batches, Adam and overfitting 33

B5 Checkpoint B: neural networks 36

Part C Text and sequences 38

C1 Tokenization: from text to IDs 39

C2 Embeddings: from IDs to vectors 41

C3 RNNs and the seq2seq bottleneck 43

C4 Attention before Transformers 45

C5 Checkpoint C: text and sequences 47

Part D Attention, one piece at a time 48

D1 Attention as a weighted average 49

D2 From scores to weights: one query by hand 51

D3 Why queries, keys and values are separate 53

D4 The matrix form and the $\sqrt{d_k}$ scale 55

D5 Multi-head attention 58

D6 Masks: padding and causal 61

D7 Positional encoding 64

D8 Checkpoint D: attention from scratch 67

Part E The Transformer 69

E1 Feed-forward, residual connections and layer normalization 70

E2 The encoder block and stack 73

E3 The decoder: masked self-attention and cross-attention 75

E4 The output layer: from vectors to next-token probabilities 78

E5 One sentence through the whole model: a shape trace 80

E6 How the Transformer is trained 82

E7 How it generates: greedy decoding and beam search 85
E8 The whole architecture, and how to read the paper 87

Part G After the paper 90
G1 Encoder-only, decoder-only and encoder–decoder 91
G2 What modern models changed, and what they kept 93
G3 Pretraining, fine-tuning and instruction tuning 95

Appendix A · Notation and glossary 97

Appendix B · Solutions to exercises 100

Appendix C · Answers to self-assessment and review questions 107

Appendix D · References 109

PART 0

Orientation

Where this book is going, and where you should start.

0.1 The destination: the whole Transformer in one picture

0.2 Readiness check: where should you start?

CHAPTER 0.1 · Part 0 · Orientation

The destination: the whole Transformer in one picture

About 10 minutes

Goal: See the complete machine before studying its parts, so every later chapter has an obvious place to fit.

The original Transformer (Vaswani et al., 2017) was built to translate. Give it the German sentence *Ich mag Tee* and it produces *I like tea*, one token at a time. Below is the entire model. Don't try to understand it yet. Each box is labelled with the chapter that explains it.

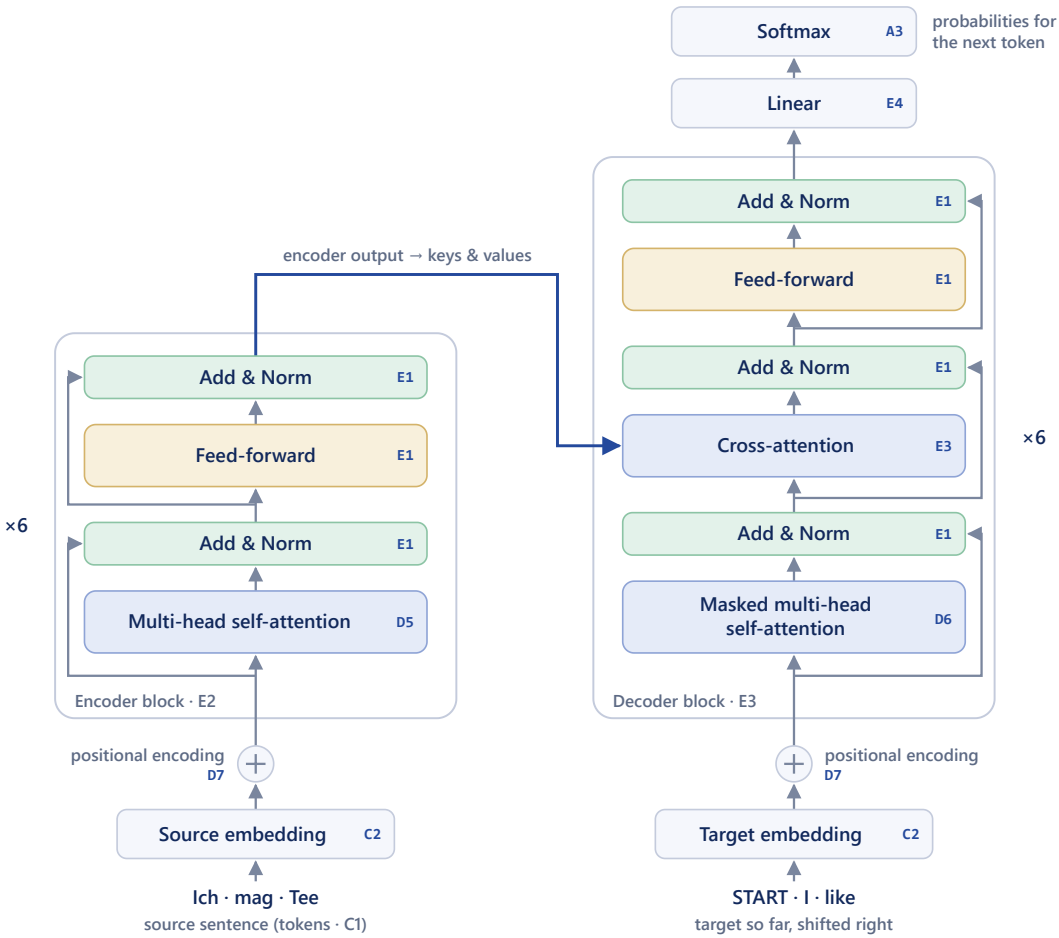


Figure 0.1.1. Flow runs bottom to top. Grey lines that bend around a box are **residual connections** (E1). The blue line carries the encoder's output into every decoder block.

Read the picture in five steps

1. **Text becomes vectors.** Each sentence is split into tokens (C1). Each token looks up a learned vector (C2), and a position signal is added to it (D7).

2. **The encoder builds context.** Six identical blocks repeat two operations. *Self-attention* lets each source token gather information from the others (D1–D5). A *feed-forward* layer then transforms each token on its own (E1).
3. **The decoder reads the target so far.** Its self-attention is *masked* so that a position cannot peek at later target words (D6).
4. **Cross-attention consults the source.** Each target position asks the encoder output which source words matter right now (E3).
5. **Scores become a prediction.** A linear layer and softmax turn the final vector into a probability for every word in the vocabulary (E4). The model picks a word, appends it, and repeats (E7).

The running example

The whole book uses one sentence pair, so you can compare each new ingredient with what came before:

Role	Tokens	Toy vectors used in calculations
Source (German)	Ich · mag · Tee	used in the encoder and cross-attention chapters
Target (English)	I · like · tea	I = [1,0], like = [0,1], tea = [1,1]

Real models use 512 numbers per token. Two numbers let you do every calculation by hand and still see the mechanism. Wherever real sizes matter, the chapter gives them too.

How the book is organized

Part	After it you can…
A · Math	compute dot products, matrix products, softmax, simple derivatives and normalization by hand
B · Neural networks	explain a forward pass, a loss, a gradient and one training step
C · Text and sequences	turn a sentence into a matrix of vectors and explain why RNNs struggled
D · Attention	calculate scaled dot-product attention, with heads, masks and positions, from scratch
E · The Transformer	trace a sentence through every layer, and explain how the model is trained and generates text
G · After the paper	place BERT, GPT and modern LLMs relative to the 2017 design

How each chapter works

Each chapter introduces **one new idea**. Under its title, the chapter lists the earlier chapters it relies on, so you know what to revisit if something feels unclear. Each chapter ends with a summary, exercises at two levels and suggestions for further reading:

- **Compute**: work a number out by hand.
- **Explain**: say why something works, in your own words.

Worked solutions to every exercise are in Appendix B. Parts B, C and D each end with a review chapter, and the answers to its questions are in Appendix C. A good rule: move on when you can answer the Explain exercises without looking back.

Two ways through

Full path: read the chapters in order. **Fast path** (you already know basic machine learning): take the self-assessment in Chapter 0.2. It tells you which chapters to review, after which you can start at Part C or Part D.

CHAPTER SUMMARY

- The Transformer is an encoder (reads the source) plus a decoder (writes the target), joined by cross-attention.
- Both stacks repeat the same pattern: attention, then a feed-forward layer, each wrapped in a residual connection and normalization.
- Every box in the diagram has one chapter. By E8 you will be able to explain all of them.

Next: find out where you should start.

Readiness check: where should you start?

About 10 minutes

Goal: Find the gaps in your background in ten minutes, so you spend time only where you need it.

Answer honestly, and guess only if you would guess in real life. Each question maps to one chapter. Check your answers in Appendix C: the earliest chapter behind any question you missed is where you should start.

Self-assessment questions

Q1 What is [2, 3] · [4, 1]?

a. 11
b. [8, 3]
c. 14
d. I don't know

Q2 Which pair of vectors points in the most similar direction?

a. [1, 0] and [−1, 0]
b. [1, 0] and [0, 1]
c. [1, 1] and [2, 2]
d. I don't know

Q3 A (5, 8) matrix is multiplied by an (8, 3) matrix. What is the result's shape?

a. (8, 8)
b. (5, 3)
c. (3, 5)
d. I don't know

Q4 softmax([2, 1, 0]) is approximately…

a. [0.67, 0.24, 0.09]
b. [0.67, 0.33, 0]
c. [1, 0, 0]
d. I don't know

Q5 For $f(w) = w^2$, what is the slope (derivative) at $w = 3$?

a. 3
b. 6
c. 9
d. I don't know

Q6 What is the (population) variance of the numbers [2, 4]?

a. 1
b. 2
c. 3
d. I don't know

Q7 Why does a neural network need a nonlinear activation such as ReLU?

a. It makes training faster
b. Without it, stacked layers collapse into one linear function
c. It turns outputs into probabilities
d. I don't know

Q8 A classifier gives the correct class probability 0.9 in one case and 0.1 in another. Cross-entropy loss is…

a. lower for 0.9
b. lower for 0.1
c. the same for both
d. I don't know

Q9 The gradient of the loss with respect to a weight w is −16. What will gradient descent do?

a. Decrease w
b. Increase w
c. Set w to −16
d. I don't know

Q10 What does an embedding layer do?

a. Looks up a learned vector for each token ID
b. Splits text into tokens
c. Converts scores into probabilities
d. I don't know

Q11 What is the main reason RNNs are hard to parallelize over a sequence?

a. They have too many parameters
b. They cannot use embeddings
c. Each step needs the previous step's hidden state
d. I don't know

Answers and explanations are in Appendix C.

CHAPTER SUMMARY

- Each wrong or "I don't know" answer points to one short chapter. Fixing these gaps now is much cheaper than discovering them in Part D.
- Come back and retake the check after Part B. Every answer should then feel obvious.

Next: keep the notation sheet handy. It's a reference you'll return to.

PART A

Math you'll actually use

Five small tools that attention is built from: dot products, matrices, softmax, derivatives and normalization.

A1 Vectors and the dot product as similarity

A2 Matrices, shapes and transpose

A3 Exponentials, logarithms and softmax

A4 Derivatives and the chain rule

A5 Mean, variance and normalization

Vectors and the dot product as similarity

About 15 minutes

Goal: Compute a dot product and read it as a match score. This is the exact operation attention uses to decide where to look.

Attention's first job is to answer one question for every pair of tokens: *how well does this one match that one?* The Transformer answers it with a single operation, the dot product. If you understand this chapter, half of the attention formula is already familiar.

A vector is a list of numbers, and also an arrow

A **vector** is an ordered list of numbers, such as a = [2, 1]. With two numbers you can draw it as an arrow from the origin to the point (2, 1). Its **length** follows Pythagoras: $|a| = \sqrt{(2^2 + 1^2)} = \sqrt{5} \approx 2.236$.

Real token vectors have 512 numbers. We can't draw those, but every idea here works the same way in 512 dimensions.

The dot product: multiply matching entries, then add

$$a \cdot b = a_1b_1 + a_2b_2 + \ldots + a_nb_n$$

$$[2, 1] \cdot [1, 2] = 2\times1 + 1\times2 = \mathbf{4}$$

The result is a single number, not a vector. You'll meet this same calculation in B1 under another name: a neuron's weighted sum is a dot product between inputs and weights.

What the number means

The dot product has a second, geometric formula: $\mathbf{a \cdot b = |a|\,|b| \cos \theta}$, where θ is the angle between the arrows. The cosine is 1 when the arrows point the same way, 0 when they are perpendicular and −1 when they are opposite. So the sign and size of the dot product tell you how aligned two vectors are:

Arrows	Example	Dot product	Reading
Same direction	[1,0] · [1,0]	1	strong match
Perpendicular	[1,0] · [0,1]	0	no alignment
Opposite	[1,0] · [−1,0]	−1	anti-match

Dividing by both lengths removes the effect of size and leaves only direction. This is called **cosine similarity**, and it always lies between −1 and 1. For [2,1] and [1,2] it is $4 \div (\sqrt{5} \times \sqrt{5}) = \mathbf{0.8}$.

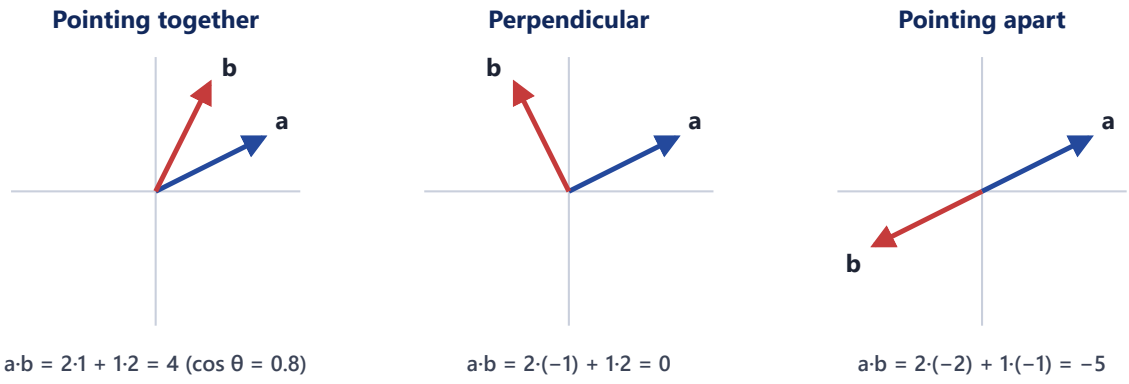


Figure A1.1. The same vector a = [2, 1] against three choices of b. The sign of the dot product tracks the angle between the arrows.

Meet the running example

From here on, the target sentence *I like tea* uses three toy vectors: I = [1,0], like = [0,1], tea = [1,1]. Here is every dot product between them:

·	I [1,0]	like [0,1]	tea [1,1]
I	1	0	1
like	0	1	1
tea	1	1	2

Two things are worth noticing. First, "like" matches "tea" exactly as well as it matches itself (both scores are 1). Second, tea · tea = 2 is larger than like · like = 1, even though each vector matches itself perfectly in direction. The raw dot product rewards *length* as well as direction.

WHY THIS MATTERS LATER

In D2 you'll compute attention for the token "like". Its first step is exactly the middle row of this table: scores [0, 1, 1]. Attention uses the raw dot product, scaled down a little (D4), rather than cosine similarity. So a vector's length affects how strongly it matches, and training learns to use that.

COMMON MISTAKE

"A high dot product means two words mean the same thing." Not by itself. The dot product measures geometric alignment. It becomes meaningful only because training arranges the vectors so that useful matches end up aligned.

CHAPTER SUMMARY

- Dot product = multiply matching entries and add. The result is one number.
- It is large when vectors align, zero when they are perpendicular and negative when they are opposed: $a \cdot b = |a||b|\cos\theta$.
- Attention scores are dot products between token vectors. Length matters as well as direction.

Exercises

A1.1 COMPUTE

$[3, -1] \cdot [2, 4] = ?$ Are these vectors pointing roughly together, apart, or at right angles?

A1.2 EXPLAIN

Why is tea · tea larger than like · like, when both vectors match themselves perfectly in direction?

Solutions are in Appendix B.

FURTHER READING

- 3Blue1Brown, Essence of Linear Algebra. The chapters on vectors and on dot products build geometric intuition fast.
  `www.3blue1brown.com/topics/linear-algebra`

Next: compute many dot products at once with matrices.

Matrices, shapes and transpose

Prerequisites: A1 Dot product as similarity · About 20 minutes

> **Goal:** Read the shapes of arrays and predict the result of a matrix product. You'll use this skill in every attention calculation.

A sentence is several token vectors, and attention compares every token with every other token. Writing those dot products one by one would be slow and error-prone. A matrix lets us compute them all in one operation.

Matrices and shapes

A **matrix** is a rectangle of numbers. In this book each *row* is a token position and each *column* is a feature. The running example stacks its three token vectors into one matrix:

```
X = [[1, 0],   ← I
     [0, 1],   ← like
     [1, 1]]   ← tea        shape (3, 2): 3 tokens, 2 features
```

The **shape** (rows, columns) is the first thing to check in any calculation. A real model's X for a 7-token sentence has shape (7, 512).

Matrix multiplication is many dot products

To multiply A by B, take the dot product of each *row of A* with each *column of B*. The entry in row i, column j of the result is row i of A · column j of B.

```
X = [[1, 2],    W = [[1, 0],
     [3, 4]]         [0, 2]]

XW = [[1×1 + 2×0, 1×0 + 2×2],     = [[1, 4],
      [3×1 + 4×0, 3×0 + 4×2]]        [3, 8]]
```

Here W keeps the first feature and doubles the second. For the product to exist, the **inner dimensions must match**, because each dot product needs equal-length vectors. The outer dimensions give the result's shape:

```
(3 × 4) @ (4 × 2) → (3 × 2)
```

Read this as: three tokens go in with 4 features each, and three tokens come out with 2 features each. Multiplying by a weight matrix changes the number of features *per token*. It never changes the number of tokens.

Transpose: turn rows into columns

The **transpose** X^T swaps rows and columns, so a (3, 2) matrix becomes (2, 3). Why does this matter? Multiply X by its own transpose:

```
X Xᵀ = (3 × 2) @ (2 × 3) → (3 × 3)

     = [[1, 0, 1],
        [0, 1, 1],
        [1, 1, 2]]
```

That is exactly the table of all dot products from A1, computed in one step. Row i, column j is token i · token j. Attention's formula contains $\mathbf{QK^T}$, and it does the same thing: every query against every key, in one product.

Two more pieces of notation

Elementwise operations. Multiplying two same-shaped matrices *entry by entry* (written A ⊙ B) is a different operation from matrix multiplication. When a formula writes AB or A·B with matrices, it means the row-by-column product.

Broadcasting. Adding a single vector to every row of a matrix is written X + b. For example, [[1,0],[0,1]] + [10, 20] = [[11,20],[10,21]]. Bias terms and positional encodings (D7) are added this way.

Tensors. A tensor is an array with any number of dimensions. A batch of 2 sentences, each with 3 tokens of 4 features, has shape (2, 3, 4): read it as (examples, tokens, features). Matrix products act on the last two dimensions, separately for each example.

CHAPTER SUMMARY

- (a × b) @ (b × c) → (a × c). The inner dimensions must match.
- A weight matrix changes the features per token, never the number of tokens.
- $X X^T$ produces every pairwise dot product at once. QK^T in attention does the same.

Exercises

A2.1 COMPUTE

X has shape (5, 8) and W has shape (8, 3). What shape is XW? What about W^T?

A2.2 COMPUTE

Q has shape (10, 64). What shape is QQ^T, and what does each entry mean?

A2.3 EXPLAIN

Why can't you multiply a (3, 2) matrix by another (3, 2) matrix, and what one change makes it possible?

Solutions are in Appendix B.

FURTHER READING

- 3Blue1Brown, matrix multiplication as composition. Useful if you want the geometric view.
  www.3blue1brown.com/topics/linear-algebra

Next: turn a row of scores into weights that add up to 1.

Exponentials, logarithms and softmax

Prerequisites: A1 Dot product as similarity · About 20 minutes

Goal: Turn any list of scores into positive weights that sum to 1, and predict how scaling the scores changes those weights.

Dot products give raw scores such as [0, 1, 1] or [−2.3, 4.1, 0.7]. Scores like these can be negative and don't add up to anything in particular. To use them as "how much of each", we need positive weights that sum to 1. The Transformer does this conversion in two places: inside attention (D2) and at the very end, where it produces word probabilities (E4).

A quick refresher: e, exp and ln

e ≈ 2.718 is a mathematical constant. **exp(x) = e^x** is always positive and grows quickly: exp(0) = 1, exp(1) ≈ 2.718, exp(2) ≈ 7.389, exp(−1) ≈ 0.368. Two properties matter here: exp(a + b) = exp(a) × exp(b), and a bigger input always gives a bigger output.

ln (the natural logarithm) undoes exp: ln(e^x) = x. Some values: ln(1) = 0, ln(0.5) ≈ −0.693, and ln of a number between 0 and 1 is always negative. Cross-entropy loss (B2) is −ln(p), so you'll see ln again soon.

Softmax in two steps

$$\text{softmax}(s)_i = \exp(s_i) \,/\, \Sigma_j \exp(s_j)$$

WORKED EXAMPLE

Scores s = [2, 1, 0].

```
Step 1, exponentiate:  [e², e¹, e⁰] ≈ [7.389, 2.718, 1.000]   total ≈ 11.107
Step 2, divide by total: [0.665, 0.245, 0.090]                 sum = 1
```

The largest score gets the largest share, but the others still get something. That is the "soft" in softmax. A hard max would give [1, 0, 0].

Four properties you will rely on

1. **Positive and sums to 1.** The outputs can be used directly as weights or probabilities.
2. **Order is preserved.** A higher score always gets a higher weight.
3. **Only differences matter.** Adding the same number to every score changes nothing: softmax([12, 11, 10]) = softmax([2, 1, 0]). Code exploits this by subtracting the maximum first, which avoids overflow.
4. **Scale controls sharpness.** Multiplying the scores spreads them apart and concentrates the weights. Dividing them flattens the weights.

Scores	softmax	Effect
[1, 0.5, 0] (÷2)	[0.506, 0.307, 0.186]	flatter
[2, 1, 0]	[0.665, 0.245, 0.090]	baseline
[4, 2, 0] (×2)	[0.867, 0.117, 0.016]	sharper

Property 4 is the reason attention divides its scores by $\sqrt{d_k}$ (D4). It is also how "temperature" works when a chatbot samples text (E7).

COMMON MISTAKE

"Softmax picks the maximum." It doesn't pick anything. It gives every option a share. Attention depends on this: a token can take 50% of its information from one word and 30% from another.

CHAPTER SUMMARY

- Softmax: exponentiate, then divide by the total. The outputs are positive and sum to 1.
- Only differences between scores matter. Bigger differences give sharper weights.
- The Transformer uses softmax for attention weights and for next-token probabilities.

Exercises

A3.1 COMPUTE

softmax([0, 0]) and softmax([0, ln 3]). Hint: e^(ln 3) = 3.

A3.2 EXPLAIN

Why does subtracting the maximum score before exponentiating not change the result?

Solutions are in Appendix B.

Next: the calculus needed to see how a model learns, kept to the essentials.

Derivatives and the chain rule

About 20 minutes

Goal: Read a derivative as "how much the output moves when I nudge the input", and use the chain rule to derive the gradient formula that B3 uses.

Training a network means adjusting millions of numbers so that a loss goes down. To do that we need, for each number, the answer to one question: *if I nudge this parameter up a little, does the loss go up or down, and by how much?* That answer is a derivative.

A derivative is a slope

For $f(w) = w^2$, nudge w from 3 to 3.01. The output moves from 9 to 9.0601, so the rate of change is $0.0601 \div 0.01 \approx 6.01$. Make the nudge smaller and the rate approaches exactly **6**. We write $f'(3) = 6$, or $df/dw = 6$ at $w = 3$.

The sign tells you the direction: positive means increasing w increases f, and negative means increasing w decreases f. The size tells you how sensitive f is to w.

Three rules cover almost everything here

Rule	Example	Derivative
Power	w^n	$n \cdot w^{n-1}$ (so $w^2 \to 2w$)
Constant multiple	$c \cdot w$	c (so $3w \to 3$)
Sum	$f(w) + g(w)$	$f'(w) + g'(w)$

Also: $\mathrm{ReLU}(z) = \max(0, z)$ has slope 1 when $z > 0$ and slope 0 when $z < 0$. At exactly $z = 0$ the slope is undefined, and frameworks simply choose 0 or 1.

Many inputs: partial derivatives and the gradient

A loss depends on many parameters. A **partial derivative** $\partial L/\partial w_1$ is the slope with respect to w_1 while every other parameter is held fixed. Collect all of them into one vector and you have the **gradient**. It points in the direction that increases L fastest, so we step the opposite way.

The chain rule

When one quantity affects another through an intermediate step, multiply the slopes along the chain:

```
if L depends on u, and u depends on w:   dL/dw = dL/du × du/dw
```

WORKED EXAMPLE: THE GRADIENT USED IN B3

A one-weight model predicts wx, and its loss is squared error $L = (wx - y)^2$. Introduce the intermediate $u = wx - y$, so that $L = u^2$.

```
dL/du = 2u                (power rule)
du/dw = x                 (constant multiple; y doesn't depend on w)
dL/dw = 2u · x = 2(wx - y) · x
```

With x = 2, y = 4 and w = 0 this gives $2(0 - 4)(2) = \mathbf{-16}$. The negative sign means increasing w lowers the loss. B3 uses exactly this number.

A deep network is a long chain: weight → hidden value → next hidden value → … → loss. **Backpropagation** (B3) is the chain rule applied systematically from the loss backward, reusing each intermediate product rather than recomputing it.

How much calculus do you need? In practice nobody differentiates a Transformer by hand: deep-learning frameworks compute every gradient automatically (automatic differentiation). You need to know what a gradient *means* and why the chain rule lets it flow back through many layers. Residual connections (E1) are designed to help exactly that flow.

CHAPTER SUMMARY

- A derivative is a slope: how far the output moves per small nudge of the input. Its sign gives the direction.
- The gradient collects the slopes for every parameter. Training steps against it.
- Chain rule: multiply the slopes along a chain of dependencies. Backpropagation is that rule applied efficiently.

Exercises

A4.1 COMPUTE

$f(w) = 3w^2 + 2w$. What is $f'(w)$, and what is its value at w = 1?

A4.2 COMPUTE

$L = (wx - y)^2$ with x = 3, y = 3 and w = 2. What is dL/dw, and should w go up or down?

A4.3 EXPLAIN

In your own words, why does the chain rule multiply the slopes rather than add them?

Solutions are in Appendix B.

FURTHER READING

- 3Blue1Brown, Essence of Calculus. Derivatives and the chain rule, visually.
  `www.3blue1brown.com/topics/calculus`

Next: mean and variance, needed for layer normalization and for the $\sqrt{d_k}$ in attention.

Mean, variance and normalization

Prerequisites: A1 Dot product as similarity · About 15 minutes

Goal: Compute a mean, variance and standard deviation, normalize a vector, and see why a dot product's typical size grows like $\sqrt{d}$.

Two pieces of the Transformer are really statistics. Layer normalization (E1) recenters and rescales every token vector. The $\sqrt{d_k}$ in the attention formula (D4) exists because dot products get bigger as vectors get wider. Both need just three quantities.

Mean, variance and standard deviation

WORKED EXAMPLE

Values [2, 4, 6, 8].

```
mean μ = (2 + 4 + 6 + 8) / 4 = 5
deviations from the mean: [−3, −1, 1, 3]
variance σ² = mean of squared deviations = (9 + 1 + 1 + 9) / 4 = 5
standard deviation σ = √5 ≈ 2.236
```

The variance measures spread in squared units. The standard deviation is back in the original units: values typically sit about σ away from the mean. (This is the *population* variance, dividing by n. That is what layer normalization uses.)

Normalizing: mean 0, spread 1

Subtract the mean, then divide by the standard deviation:

```
(x − μ) / σ:  [2, 4, 6, 8] → [−1.342, −0.447, 0.447, 1.342]
```

The result has mean 0 and standard deviation 1, whatever scale the input had. Layer normalization does exactly this to each token's features, then lets the model rescale with learned parameters. It keeps numbers in a predictable range as they pass through dozens of layers.

Why dot products grow with width

Here is a useful fact: when you add up *independent* random quantities, their variances add. A dot product $q \cdot k = q_1k_1 + q_2k_2 + \ldots + q_dk_d$ is a sum of d terms. If each q_i and k_i is random with mean 0 and variance 1, each product q_ik_i has variance 1, so the sum has variance d.

Width d	Variance of q · k	Typical size (std = √d)
2	2	≈ 1.4
64	64	8
512	512	≈ 22.6

Wider vectors give larger dot products, purely because of width. In D4 you'll see that large scores make softmax extremely peaked, and that dividing by $\sqrt{d}$ puts the scores back to a typical size of about 1. You now know where that $\sqrt{d}$ comes from.

CHAPTER SUMMARY

- Variance = the mean squared deviation. Standard deviation = its square root.
- Normalizing, $(x - \mu)/\sigma$, gives mean 0 and spread 1. Layer norm applies this to each token.
- A dot product of d random unit-variance components has typical size $\sqrt{d}$. That is why attention divides by $\sqrt{d_k}$.

Exercises

A5.1 COMPUTE

Normalize [1, 3]. Then normalize [10, 30].

A5.2 EXPLAIN

If vectors had 256 dimensions with random unit-variance entries, what typical dot-product size would you expect, and what scale factor would bring it back to about 1?

Solutions are in Appendix B.

Part A is done. Next, Part B: put these tools together into a neural network.

PART B

Neural networks

How a network computes, measures its error and learns.

B1	Neurons and layers, starting from familiar ML
B2	Measuring error: squared error and cross-entropy
B3	Gradient descent and backpropagation
B4	Training in practice: batches, Adam and overfitting
B5	Checkpoint B: neural networks

Neurons and layers, starting from familiar ML

Prerequisites: A1 Dot product as similarity, A2 Matrices, shapes and transpose · About 15 minutes

Goal: Understand weights, biases, activations and hidden layers, and see that a whole layer is one matrix multiplication.

Start with a model you already know. Suppose a simple regression model estimates delivery time as **2 × distance + 5 × stops + 10**. For 3 km and 2 stops it predicts 26 minutes. (The numbers are invented for this example.)

What turns this into a neuron?

Distance and stops are the inputs. The numbers 2 and 5 are **weights**: they control how strongly each input affects the result. The 10 is a **bias**, an adjustable offset. Training changes these parameters. Processing a new delivery changes only the inputs.

```
z = w₁x₁ + w₂x₂ + b  = w · x + b      (a dot product plus a bias, from A1)
neuron output = activation(z)
```

A neuron computes that weighted sum, then passes it through an **activation function**. The most common one is ReLU, which keeps positive values and replaces negative values with zero: ReLU(3) = 3, ReLU(−2) = 0.

Why add an activation and more layers?

A network feeds one layer's outputs into the next layer as inputs. A layer in the middle is called a **hidden layer**. It computes useful features, so we don't have to design every feature by hand.

But stacking plain weighted sums gives you nothing new. If h = 2x + 1 and y = 3h + 4, then y = 6x + 7, and the two layers collapse into one. A **nonlinear** activation such as ReLU breaks that collapse, which lets the network represent curved, conditional relationships.

A TINY FORWARD PASS

Let x = 2. The hidden neuron computes z = 0.5 × 2 − 2 = −1, and ReLU makes its output h = 0. A final linear neuron computes 3h + 1 = **1**. This is prediction, not training: no parameter changed.

A layer of neurons is one matrix multiplication

Give the delivery model a second neuron with weights [1, −1] and bias 0. Put each neuron's weights in a *column* of W:

```
x = [3, 2]          W = [[2,  1],      b = [10, 0]
                         [5, −1]]

xW + b = [3×2 + 2×5,  3×1 + 2×(−1)] + [10, 0] = [16, 1] + [10, 0] = [26, 1]
```

Both neurons are computed at once: 26 from the first, as before, and 1 from the second. With a batch of deliveries stacked as rows of X, the same line computes every neuron for every delivery: **XW + b**. This is called a *linear layer* (also known as a dense or fully connected layer), and it is the most common operation in a Transformer.

```
X: (n, d_in)   @   W: (d_in, d_out)   +   b: (d_out)   →   (n, d_out)
```

A Transformer is a neural network with a particular arrangement of layers. Its feed-forward sublayer (E1) is exactly two linear layers with ReLU between them, widening each token from 512 to 2048 features and back.

CHAPTER SUMMARY

- Neuron = dot product of inputs and weights, plus a bias, passed through an activation.
- Nonlinear activations are what make depth useful.
- A layer of neurons is XW + b: one matrix multiplication for all neurons and all examples.

Exercises

B1.1 COMPUTE

In the tiny network, change x to 6. What is the prediction?

B1.2 COMPUTE

A linear layer maps each token from 512 features to 2048. What is the shape of W, and how many parameters do W and b have together?

B1.3 EXPLAIN

Why would a 10-layer network with no activation functions be no more powerful than a single linear layer?

Solutions are in Appendix B.

FURTHER READING

- 3Blue1Brown, Neural Networks, chapter 1. Neurons and layers, visually.
  `www.3blue1brown.com/topics/neural-networks`

Next: measure how wrong a prediction is.

Measuring error: squared error and cross-entropy

Prerequisites: A3 Exponentials, logs and softmax · About 20 minutes

> **Goal:** Compute the loss for a number prediction and for a classification, and understand the loss the Transformer actually minimizes.

A network can't improve unless it has one number that says how bad its current prediction is. That number is the **loss**. Training is the search for parameters that make the loss small on average.

Predicting a number: squared error

For a numerical target, a simple choice is (prediction − target)^2. Predicting 26 minutes when the truth is 30 gives a loss of 16. Squaring makes every error positive and punishes large errors heavily.

Predicting a category: logits → softmax → probability

For classification, the network outputs one raw score per class. These scores are called **logits**. Softmax (A3) turns them into probabilities. Suppose a classifier for {cat, dog, bird} outputs logits [2, 1, 0]:

```
softmax([2, 1, 0]) = [0.665, 0.245, 0.090]     → "cat" with probability 0.665
```

Cross-entropy: −ln(probability of the right answer)

```
loss = -ln(p_correct)
```

Correct class	p_correct	Loss
cat	0.665	−ln 0.665 ≈ 0.408
bird	0.090	−ln 0.090 ≈ 2.408
(perfect)	1.000	0

A confident, correct model gets a loss near 0. A model that gave the right answer little probability gets a large loss. The loss keeps growing as p approaches 0, so confidently wrong answers are punished most.

A REMARKABLY SIMPLE GRADIENT

For softmax followed by cross-entropy, the gradient of the loss with respect to the logits is just **p − y**, where y is the one-hot vector for the correct class. If the correct class is cat, then y = [1, 0, 0]:

```
p − y = [0.665 − 1, 0.245, 0.090] = [−0.335, 0.245, 0.090]
```

Gradient descent moves against this: it raises the cat logit and lowers the others in proportion to the probability each one wrongly took. You don't need to derive this formula, but it explains why this pairing trains so well.

The Transformer's loss

Predicting the next token is classification with one class per vocabulary entry, about 37,000 of them in the paper's English–German setup. At every target position the model outputs 37,000 logits. The loss is −ln of the probability given to the correct next token, averaged over all positions.

Papers often report **perplexity** = e^(average loss). It's roughly "how many words the model is torn between". A loss of 1.609 gives perplexity e^1.609 ≈ 5. The original paper reports perplexity on its development set.

CHAPTER SUMMARY

- Loss is a single number measuring how bad the predictions are. Training lowers it.
- Classification: logits → softmax → cross-entropy = −ln(p_correct).
- The Transformer is a classifier over the vocabulary, applied at every target position.

Exercises

B2.1 COMPUTE

The model gives the correct token probability 0.8. What is the loss? What if it gives 0.2?

B2.2 EXPLAIN

Why is cross-entropy better than "1 if wrong, 0 if right" for training?

Solutions are in Appendix B.

Next: use the loss's gradient to improve the parameters.

Gradient descent and backpropagation

Prerequisites: A4 Derivatives and the chain rule, B2 Loss functions · About 25 minutes

Goal: Perform one training update by hand, and follow backpropagation through a two-layer network.

A forward pass produces a prediction with the current weights. Training asks a different question: how should those weights change so that the loss goes down? The answer is to compute the gradient and take a small step against it. Then repeat.

One complete update

WORKED EXAMPLE

The model predicts $w \times x$, with $x = 2$, target 4, and w starting at 0. The prediction is 0 and the loss is $(0 - 4)^2 = 16$. From A4, the gradient is $2(wx - y) \cdot x = 2(0 - 4)(2) = -16$.

```
new w = old w − learning_rate × gradient
      = 0 − 0.1 × (−16) = 1.6
```

The new prediction is 3.2 and the new loss is 0.64. The negative gradient told us to increase w, and the learning rate controlled how far.

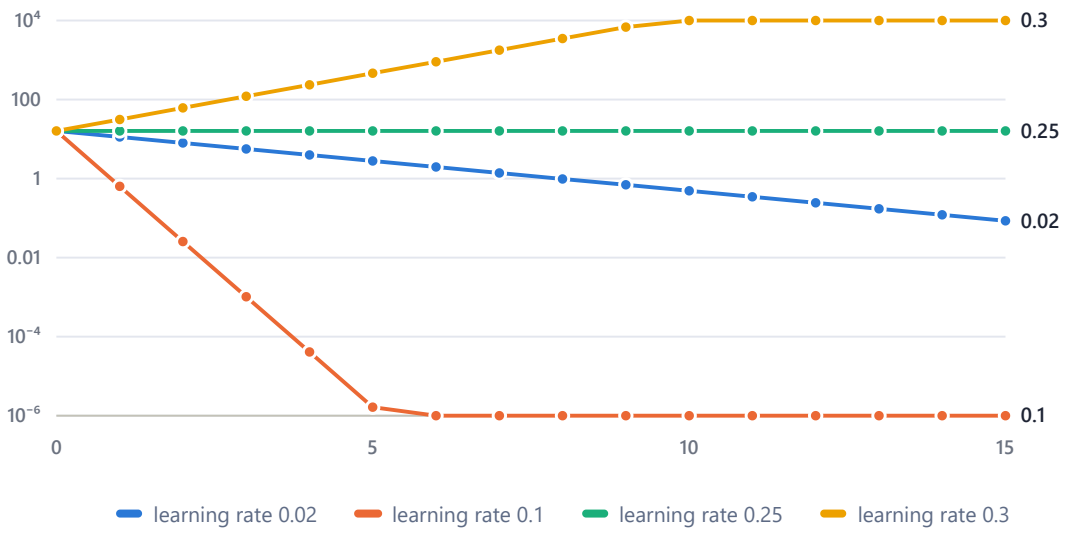


Figure B3.1. Loss over 15 gradient-descent updates for the model $w \cdot x$ ($x = 2$, target 4, start $w = 0$), on a log scale. 0.02 is slow, 0.1 is fast, 0.25 bounces between $w = 0$ and $w = 4$ forever, and 0.3 diverges. (The axis is clipped at both ends: 0.1 keeps shrinking below 10^{-6}, and 0.3 keeps growing above 10^4.)

Backpropagation: gradients for every layer

With several layers, an early weight affects a hidden value, which affects the next layer, which affects the loss. The chain rule (A4) multiplies the local slopes along that path. **Backpropagation** applies the rule efficiently, starting at the loss and working backward. Each step reuses the product already computed for the layer above it.

BACKPROP BY HAND: TWO WEIGHTS

Network: $z = w_1 x$, $h = \text{ReLU}(z)$, $\text{prediction} = w_2 h$, $L = (\text{prediction} - y)^2$. We set $x = 2$, $y = 2$, $w_1 = 0.5$, $w_2 = 1$.

```
Forward:  z = 1,  h = 1,  prediction = 1,  L = (1 − 2)² = 1

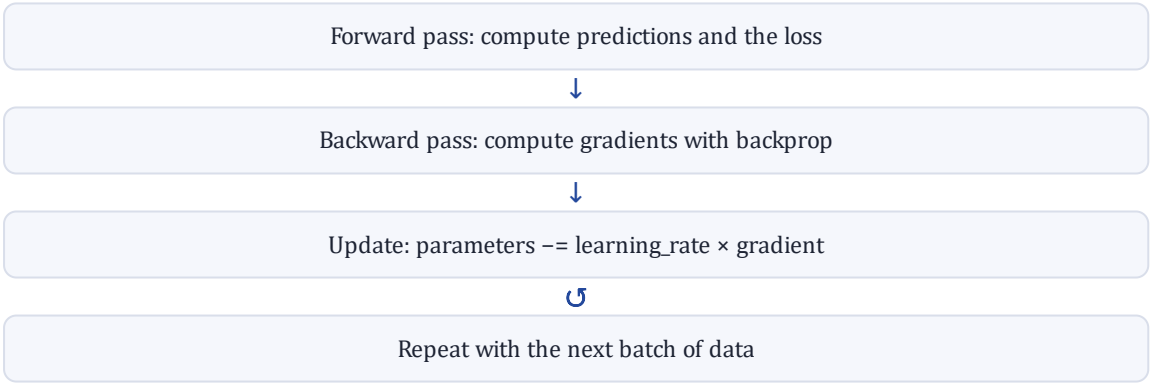
dL/dpred = 2(pred − y) = −2
dL/dw₂   = dL/dpred × h  = −2 × 1 = −2
dL/dw₁   = dL/dpred × w₂ × ReLU′(z) × x = (−2)(1)(1)(2) = −4
```

$\text{ReLU}'(z) = 1$ because z is positive. With learning rate 0.1: w_1 becomes $0.5 + 0.4 = 0.9$, and w_2 becomes $1 + 0.2 = 1.2$. The new forward pass gives $h = 1.8$, prediction 2.16 and loss 0.0256, much better.

TWO COMMON CONFUSIONS

Backpropagation *computes* gradients. The optimizer *uses* them to update the weights. These are separate steps. Also, compute every gradient from the same old weights before updating any of them.

The training loop, and inference



During **inference**, when the trained model is in use, only the forward pass runs, and the parameters stay fixed. Translating a new sentence changes the inputs and intermediate values, never W_Q or any other learned matrix.

CHAPTER SUMMARY

- Update rule: $w \leftarrow w - \text{learning_rate} \times \partial L/\partial w$.
- Backpropagation is the chain rule run backward from the loss. It gives every parameter's gradient in one pass.
- The learning rate has to be tuned: too small is slow, too large oscillates or diverges.

Exercises

B3.1 COMPUTE

Continue the one-weight example: $w = 1.6$, $x = 2$, $y = 4$, learning rate 0.1. What is the next w?

B3.2 EXPLAIN

In Figure B3.1, why does learning rate 0.25 bounce forever?

Solutions are in Appendix B.

FURTHER READING

- 3Blue1Brown, Neural Networks, chapters 2–4. Gradient descent and backpropagation explained visually.
  `www.3blue1brown.com/topics/neural-networks`

Next: how training works in practice, with data at scale.

Training in practice: batches, Adam and overfitting

Prerequisites: B3 Gradient descent and backprop · About 20 minutes

Goal: Know the practical machinery around gradient descent that the Transformer paper relies on: mini-batches, Adam, learning-rate schedules, dropout and validation data.

B3 updated one weight on one example. The paper trained 65 million parameters on 4.5 million sentence pairs. A few practical ideas bridge that gap, and each one appears by name in the paper's training section.

Mini-batches and epochs

Computing the exact gradient over the whole dataset before every step would be far too slow. Instead, each step uses a **mini-batch**: a random sample of examples whose average gradient approximates the full one. This is *stochastic* gradient descent. One full pass over the data is an **epoch**. The paper built batches of about 25,000 source tokens and 25,000 target tokens, grouping sentences of similar length so that little padding was wasted.

Adam: a smarter step

Plain gradient descent uses the same learning rate for every parameter. **Adam** keeps two running averages for each parameter. One is the average gradient, called *momentum*, which smooths out noisy batches. The other is the average squared gradient, which shrinks the step for parameters whose gradients are consistently large. The paper used Adam with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\varepsilon = 10^{-9}$. Those three numbers control the two averages and prevent division by zero.

Learning-rate schedules and warmup

The learning rate doesn't have to be constant. Early in training the Adam statistics are unreliable and the gradients can be wild, so the paper **warms up**: it increases the learning rate linearly over the first 4,000 steps, then decays it slowly. E6 plots the exact formula.

Overfitting and how to detect it

A model can memorize its training data and still fail on new sentences. This is **overfitting**. To catch it, keep data aside: a *validation* set to monitor during training, and a *test* set touched only for the final result. When the training loss keeps falling but the validation loss rises, the model is memorizing.

Regularization: dropout, and a preview of label smoothing

Dropout randomly zeroes a fraction of activations at each training step, so the network can't rely on any single path. With rate 0.1, about 10% of values become 0, and the survivors are multiplied by 1/0.9 so that the average stays the same. At inference dropout is switched off. The paper applies dropout of 0.1 to every sublayer output and to the embeddings.

DROPOUT ON ONE VECTOR

Rate 0.5 (large, for visibility). [2, 4, 6, 8] with a random mask [1, 0, 1, 0] → [2, 0, 6, 0] × 2 = **[4, 0, 12, 0]**. On average over many masks, each entry still equals its original value.

Label smoothing (E6) trains against slightly softened targets so the model doesn't become overconfident. Other common tools include weight decay and early stopping.

Setting in the paper	Base model value	Idea from this chapter
Optimizer	Adam, β_1 0.9, β_2 0.98, ε 10^{-9}	adaptive steps
Batch size	≈ 25,000 source + 25,000 target tokens	mini-batches
Warmup	4,000 steps	schedule
Dropout P_drop	0.1	regularization
Label smoothing ε_ls	0.1	regularization (E6)
Training length	100,000 steps, about 12 hours on 8 P100 GPUs	—

CHAPTER SUMMARY

- Each step uses a mini-batch gradient, a noisy but cheap estimate of the full gradient.
- Adam adapts the step size per parameter. Warmup protects the fragile first steps.
- Keep validation data aside. Dropout and label smoothing fight overfitting.

Exercises

B4.1 COMPUTE

Dropout rate 0.2 is applied to [5, 5, 5, 5] with mask [1, 1, 0, 1]. What is the output?

B4.2 EXPLAIN

Training loss is 0.9 and falling, while validation loss went from 1.4 to 1.7 over the last few epochs. What is happening, and name two responses.

Solutions are in Appendix B.

FURTHER READING

- Kingma & Ba (2014), Adam. The optimizer paper. Sections 1–2 are readable.
  `arxiv.org/abs/1412.6980`
- Srivastava et al. (2014), Dropout.
  `jmlr.org/papers/v15/srivastava14a.html`

Next: a checkpoint to confirm Part B has stuck.

Checkpoint B: neural networks

Prerequisites: B1 Neurons and layers, B2 Loss functions, B3 Gradient descent and backprop, B4 Training in practice · About 15 minutes

Goal: Confirm that you can use Parts A and B without looking back. Questions mix both parts on purpose, because retrieving older material is what makes it stick.

Review questions

Q1 A linear layer maps (7, 512) inputs to (7, 2048) outputs. What shape is its weight matrix?

a. (7, 2048)
b. (2048, 512)
c. (512, 2048)
d. (7, 512)

Q2 Logits [0, 0, ln 2], and the correct class is the third. What is the cross-entropy loss?

a. 0
b. $-\ln 0.5 \approx 0.693$
c. $-\ln 0.25 \approx 1.386$
d. $\ln 2 \times 3 \approx 2.079$

Q3 Which statement about backpropagation is correct?

a. It computes gradients by applying the chain rule from the loss backward
b. It updates the weights
c. It runs during inference to adapt the model to each input
d. It replaces the loss function

Q4 Validation loss rises while training loss falls. The best description is…

a. the learning rate is too small
b. underfitting
c. a bug in softmax
d. overfitting

Q5 Multiplying all attention scores by 3 before softmax will…

a. leave the weights unchanged
b. make the weights more concentrated on the top score
c. make the weights more uniform
d. make the weights sum to 3

Q6 Random vectors with 100 unit-variance components: what is the typical size of their dot product?

a. 1
b. 100
c. 10
d. 0, exactly

Answers and explanations are in Appendix C.

Exercises

B5.1 COMPUTE

One full training step: the model predicts w·x, with x = 3, target 6, w = 1 and learning rate 0.05. Give the prediction, the loss, the gradient, the new w and the new loss.

B5.2 EXPLAIN

Out loud, in under a minute: what happens in one training step of a neural network?

Solutions are in Appendix B.

Part B is done. Next, Part C: how text becomes numbers, and why older sequence models struggled.

PART C

Text and sequences

How words become vectors, and why the sequence models before 2017 struggled.

C1 Tokenization: from text to IDs

C2 Embeddings: from IDs to vectors

C3 RNNs and the seq2seq bottleneck

C4 Attention before Transformers

C5 Checkpoint C: text and sequences

Tokenization: from text to IDs

About 15 minutes

> **Goal:** Explain what a tokenizer does, run byte-pair encoding by hand, and know the special tokens a Transformer needs.

A network can't calculate with the word "tea". Text first has to become numbers, and that happens in two separate steps. A **tokenizer** chooses the pieces and gives each one an ID. An **embedding** (C2) gives each ID a vector. This chapter covers the first step.

Tokens and IDs

A tokenizer splits text into tokens and maps each token to its index in a fixed **vocabulary**. In a deliberately simple word-level example, "I like tea" becomes ["I", "like", "tea"] with IDs [0, 1, 2]. These IDs are addresses, not quantities: token 2 is not "twice" token 1.

Why not words, and why not characters?

Whole words make the vocabulary huge, and any unseen word ("teapots", a name, a typo) becomes an unknown token. **Single characters** never meet an unknown, but sentences become very long, and the model has to learn spelling before it can learn meaning. **Subwords** sit in between. Common words stay whole, and rare words split into reusable pieces.

Byte-pair encoding (BPE), by hand

BPE builds a subword vocabulary from data. Start with characters, then repeatedly merge the most frequent adjacent pair into a new token. Take a tiny corpus of word counts: *low* ×5, *lower* ×2, *newest* ×6, *widest* ×3.

Merge	Most frequent pair	Count	Why
1	e + s → es	9	newest (6) + widest (3)
2	es + t → est	9	the same two words
3	l + o → lo	7	low (5) + lower (2)
4	lo + w → low	7	the same two words

Now tokenize a word the corpus never contained, *lowest*. Apply the merges in order: l o w e s t → l o w es t → l o w est → lo w est → **low · est**. Two meaningful pieces, and no unknown token. (Real BPE also marks word endings and runs tens of thousands of merges.)

Special tokens

Token	Job	Where you'll see it
START (BOS)	"begin generating": the decoder's first input	D6, E3, E7
END (EOS)	"this sentence is finished": generation stops here	E6, E7
PAD	filler that makes sequences in a batch the same length. Always masked out.	D6

What the paper used

For English→German, the paper used BPE with a **shared source–target vocabulary of about 37,000 tokens**. For English→French it used a 32,000-token word-piece vocabulary, a closely related method. A shared vocabulary lets the model use one embedding table for both languages (E4).

CHAPTER SUMMARY

- Tokenizer: text → tokens → vocabulary IDs. IDs are addresses, not quantities.
- Subword methods such as BPE keep common words whole and split rare ones into reusable pieces.
- START, END and PAD are special tokens with structural jobs.

Exercises

C1.1 COMPUTE

Using the four merges above, tokenize *newer* and *slow*.

C1.2 EXPLAIN

Why can't the IDs themselves be fed straight into the network as numbers?

Solutions are in Appendix B.

FURTHER READING

- Sennrich et al. (2016), BPE for neural machine translation.
  `arxiv.org/abs/1508.07909`

Next: give each ID a learned vector.

Embeddings: from IDs to vectors

Prerequisites: A2 Matrices, shapes and transpose, C1 Tokenization · About 15 minutes

Goal: Understand embedding lookup, why it is a matrix product in disguise, and why a token's first vector is not its final meaning.

We have IDs. The network needs vectors. An embedding gives each vocabulary entry its own learned vector, and that is where the running example's numbers come from.

The embedding table

Token	ID	Toy embedding (row of E)
I	0	[1, 0]
like	1	[0, 1]
tea	2	[1, 1]
coffee	3	[1, 0.8]

An **embedding table** E has one row per vocabulary entry. Looking up IDs [0, 1, 2] stacks rows 0, 1 and 2 into the (3, 2) matrix X you met in A2. These values are hand-picked for easy arithmetic. Real values start random and are learned.

The paper calls the vector width **d_model = 512**. With a 37,000-token vocabulary, E has shape (37000, 512), about 19 million numbers. A 7-token sentence becomes a (7, 512) input matrix.

A lookup is secretly a matrix product

Write token ID 2 as a **one-hot** vector: all zeros except a 1 at position 2, giving [0, 0, 1, 0]. Multiplying it by E selects row 2:

```
[0, 0, 1, 0] @ E = 0·[1,0] + 0·[0,1] + 1·[1,1] + 0·[1,0.8] = [1, 1]   ← tea's row
```

So an embedding is just a linear layer (B1) applied to one-hot inputs, and it is trained by backprop like any other weight. Libraries skip the multiplication and index the row directly, because that is far faster. Only the rows for tokens in the current batch receive gradient updates.

Similar tokens end up nearby

Training moves vectors around so that tokens used in similar ways get similar vectors. In our toy table, tea [1,1] and coffee [1,0.8] are close: their cosine similarity (A1) is about 0.99. Individual dimensions usually have no clean human label such as "drinkable". The meaning is in the geometry, not in any single number.

The starting vector is not the final meaning

"Bank" gets the same embedding row in "river bank" and in "bank loan". A useful model must represent those two occurrences differently once it has read the context. The layers above the embedding build **contextual representations** (also called hidden states), and attention is the main tool for that. The jobs divide cleanly: tokenization identifies the pieces, embeddings give learned starting points, and the network transforms those starting points using context.

Paper detail (you'll see it again in E4): the Transformer multiplies embeddings by $\sqrt{d_model} \approx 22.6$ before adding positional encodings, and shares one matrix between the source embedding, the target embedding and the final output layer.

CHAPTER SUMMARY

- Embedding = row lookup in a learned (vocabulary × d_model) table, which is mathematically a one-hot × matrix product.
- Tokens used similarly end up with similar vectors.
- The embedding is a context-free starting point. Later layers make it contextual.

Exercises

C2.1 COMPUTE

With the toy E above, which vector does the ID sequence [2, 2, 0] produce? What is the input matrix's shape?

C2.2 EXPLAIN

The same token appears twice in a sentence. Does it look up the same embedding? Must its final contextual vectors be identical?

Solutions are in Appendix B.

Next: how sequence models worked before the Transformer, and what held them back.

RNNs and the seq2seq bottleneck

Prerequisites: B1 Neurons and layers, C2 Embeddings · About 15 minutes

Goal: Describe how a recurrent encoder–decoder translates, and name the three problems the Transformer was designed to remove.

Imagine translating *Ich mag Tee*. A model must represent the source sentence, relate words that may be far apart, and generate a target sentence. Before 2017, the strongest systems did this with recurrent neural networks.

A recurrent network reads one token at a time

An RNN keeps a **hidden state** vector, a running summary. At each step it combines the current token's embedding with the previous state to produce the new state:

```
h_t = f(W_x x_t + W_h h_{t-1} + b)        (f is a nonlinearity; LSTMs and GRUs are fancier versions)
```

Think of reading while updating a running note, except that the note is a learned vector of fixed size, however long the sentence is.

Encoder–decoder (seq2seq)

For translation, one RNN (the **encoder**) reads the source, and a second RNN (the **decoder**) generates the target, starting from the encoder's final state.

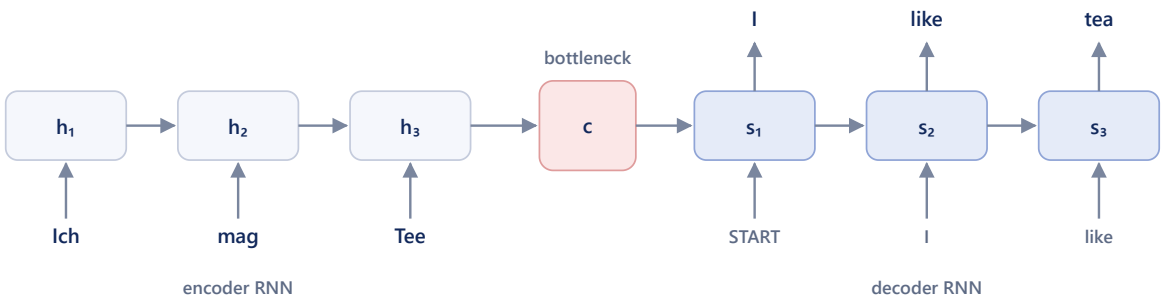


Figure C3.1. Everything the decoder knows about the source must squeeze through the single vector c.

Three problems

1. **The bottleneck.** A 50-word sentence and a 5-word sentence both have to fit into one fixed-size vector c. Long sentences lose detail.

2. **Long paths.** Information from word 1 reaches word 40 only through 39 state updates. During training, gradients must travel the same long chain back, and they tend to shrink (the *vanishing gradient* problem), so long-range dependencies are hard to learn.
3. **Sequential computation.** h_t can't be computed until h_{t−1} is done. Even when the whole sentence is known in advance, as it is in training, a GPU can't process the positions in parallel.

The next chapter shows how attention fixed problem 1. The Transformer then fixed problems 2 and 3 by removing recurrence entirely.

CHAPTER SUMMARY

- An RNN updates a hidden state one token at a time. Seq2seq uses an encoder RNN followed by a decoder RNN.
- The problems: one-vector bottleneck, long paths for information and gradients, and no parallelism across positions.

Exercises

C3.1 EXPLAIN

Why does a GPU train an RNN slowly even with a large batch of known sentences?

C3.2 EXPLAIN

Which of the three problems would get worse if the source sentences were twice as long?

Solutions are in Appendix B.

FURTHER READING

- Sutskever et al. (2014), Sequence to Sequence Learning.
  `arxiv.org/abs/1409.3215`

Next: the 2014 idea that removed the bottleneck, which is attention.

Attention before Transformers

Prerequisites: C3 RNNs and the bottleneck, A3 Exponentials, logs and softmax, A1 Dot product as similarity · About 15 minutes

Goal: See that attention was invented as an add-on to RNNs, and pin down exactly what the Transformer paper changed.

In 2014, Bahdanau, Cho and Bengio asked a simple question: why force the decoder to rely on one summary vector, when every encoder state h_1, h_2, h_3 is still available? Let the decoder *look back* at all of them, and choose differently at each step.

The mechanism

At each decoder step the model does three things:

1. **Score** every encoder state against the current decoder state.
2. **Softmax** the scores into weights (A3).
3. Take the **weighted average** of the encoder states. The result is a *context vector* made fresh for this step.

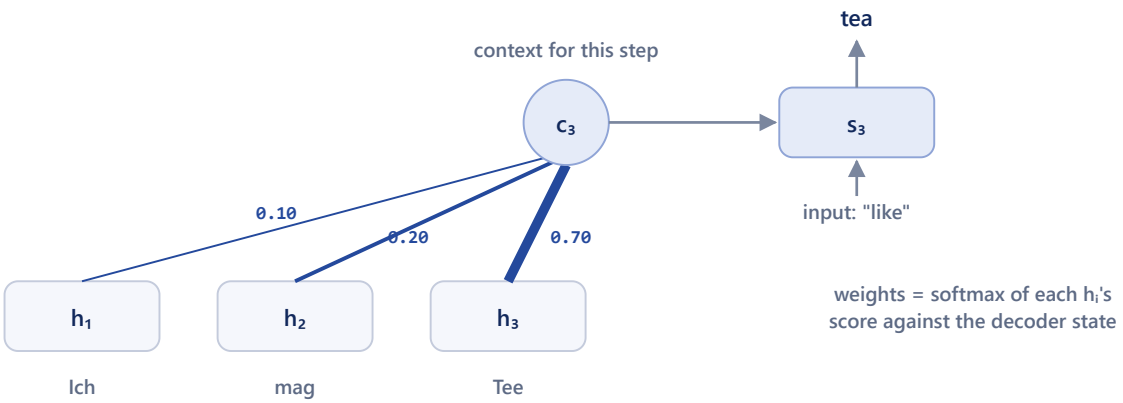


Figure C4.1. Line thickness shows the attention weight. While producing "tea", the decoder draws mostly on the state for "Tee". (The weights are illustrative.)

This removes the bottleneck. Nothing has to be squeezed into one vector any more, because each output step builds its own mixture. Translation of long sentences improved substantially. You will build exactly this weighted-average idea in D1.

What was still missing

The encoder and decoder were still RNNs. Attention connected them, but the long paths *within* each sentence and the step-by-step computation (C3, problems 2 and 3) remained.

The Transformer's move

The Transformer paper did not invent attention. Its proposal was to build the whole sequence model out of attention and drop recurrence and convolution entirely. The new ingredient that makes this possible is **self-attention**: each position in a sentence attends to the other positions in the *same* sentence. In one layer, word 1 and word 40 interact directly. When the sequence is known, as in training, every position's calculation can run in parallel.

There is a tradeoff. Full self-attention scores every pair of positions, so its cost grows with n^2 for a sequence of length n. For sentence-length inputs this is cheap. For very long documents it becomes the main cost (G2).

READ THE TITLE IN CONTEXT

"Attention Is All You Need" doesn't mean the model contains only attention. It also has embeddings, feed-forward layers, residual connections, normalization and an output projection. The title means attention is all you need *for mixing information between positions*, with no recurrence.

CHAPTER SUMMARY

- Attention (2014) = score → softmax → weighted average of states, recomputed at every decoder step.
- The Transformer's novelty is self-attention everywhere and no recurrence at all.
- The price is an n × n table of scores per layer.

Exercises

C4.1 EXPLAIN

In one sentence each, what did 2014 attention fix, and what did the 2017 Transformer add?

C4.2 COMPUTE

Encoder states $h_1 = [1, 0]$, $h_2 = [0, 1]$, $h_3 = [2, 2]$, with weights [0.1, 0.2, 0.7]. What is the context vector?

Solutions are in Appendix B.

FURTHER READING

- Bahdanau, Cho & Bengio (2014), Neural Machine Translation by Jointly Learning to Align and Translate. `arxiv.org/abs/1409.0473`

Next: a quick checkpoint, then Part D builds attention piece by piece.

Checkpoint C: text and sequences

Prerequisites: C1 Tokenization, C2 Embeddings, C3 RNNs and the bottleneck, C4 Attention before Transformers · About 10 minutes

Goal: Make sure the motivation for the Transformer is clear before building it.

Review questions

Q1 A vocabulary has 32,000 tokens and d_model = 512. What shape is the embedding table?

a. (512, 32000)
b. (32000, 512)
c. (32000, 32000)
d. (n, 512)

Q2 Why does BPE rarely produce an "unknown" token?

a. Its vocabulary contains every English word
b. It ignores rare words
c. Any word can fall back to smaller pieces, down to characters or bytes
d. It uses embeddings

Q3 What problem did 2014 attention solve for RNN translators?

a. Squeezing the whole source into one fixed vector
b. Sequential computation
c. Tokenizing rare words
d. Overfitting

Q4 Which statement about the Transformer is accurate?

a. It invented attention
b. It contains only attention layers
c. It generates all output tokens simultaneously
d. It removed recurrence and relies on self-attention to mix positions

Q5 A one-hot vector [0, 1, 0] times E = [[1,0],[0,1],[1,1]] gives…

a. [1, 1]
b. [0, 1]
c. [1, 0]
d. [0, 1, 0]

Answers and explanations are in Appendix C.

Part C is done. Next, Part D: attention, one ingredient at a time.

PART D

Attention, one piece at a time

Scaled dot-product attention, built one ingredient at a time, with heads, masks and positions.

D1	Attention as a weighted average
D2	From scores to weights: one query by hand
D3	Why queries, keys and values are separate
D4	The matrix form and the $\sqrt{d_k}$ scale
D5	Multi-head attention
D6	Masks: padding and causal
D7	Positional encoding
D8	Checkpoint D: attention from scratch

Attention as a weighted average

Prerequisites: A1 Dot product as similarity, C4 Attention before Transformers · About 15 minutes

Goal: Understand attention's output (a context-dependent weighted mixture of vectors) before worrying about where the weights come from.

In "river bank", the representation of "bank" should absorb some information from "river". Attention is the operation that gathers information from relevant positions. For this chapter, assume someone hands us the weights, and look only at what the operation does with them.

Start with a plain average, then improve it

Suppose three positions offer the vectors $v_1 = [2, 0]$, $v_2 = [0, 2]$ and $v_3 = [2, 2]$. (These are exactly twice our token vectors. D3 explains why the vectors being mixed are a separate, learned copy.) A plain average gives each position weight ⅓. But different positions should contribute different amounts, depending on which token is doing the gathering.

Attention uses a **weighted** average. With weights [0.2, 0.3, 0.5]:

```
0.2·[2,0] + 0.3·[0,2] + 0.5·[2,2]
= [0.4, 0] + [0, 0.6] + [1, 1]
= [1.4, 1.6]
```

The output is a *new vector* that blends information from all three. It isn't the list of weights, and it isn't the index of the largest weight. If the weights were [1, 0, 0], the output would simply be v_1. With [⅓, ⅓, ⅓] it would be the plain average.

The weights must depend on who is asking

"Bank" needs a different mixture from "river", even within the same sentence. So each position computes its own weights: compatibility scores between itself and every candidate position, turned into weights by softmax. That is the next chapter. The result is one mixture per position, and every position is updated at the same time.

The "river bank" story explains the motivation. It isn't a rule anyone writes into the network. Useful patterns are learned from the training objective.

Keep these two things separate

Attention weights say how much each position contributes to one mixture. They are non-negative and sum to 1. The **attention output** is the weighted sum of vectors. A large weight isn't a claim that a word is "important" in a human sense, and it isn't a complete explanation of the model's decision.

CHAPTER SUMMARY

- Attention output = Σ $\text{weight}_i \times \text{vector}_i$, a blend whose width equals the vectors' width.
- Every position gets its own weights, so every position gets its own blend.
- Weights describe the mixture. The output is the information actually passed on.

Exercises

D1.1 COMPUTE

Mix [4, 0] and [0, 8] with weights 0.75 and 0.25.

D1.2 EXPLAIN

Compare with C4: what plays the role of the "encoder states" and the "context vector" here?

Solutions are in Appendix B.

Next: compute the weights from dot products and softmax.

From scores to weights: one query by hand

Prerequisites: A1 Dot product as similarity, A3 Exponentials, logs and softmax, D1 Attention as a weighted average · About 20 minutes

Goal: Compute a complete (simplified) attention output for one token using only tools from Part A: dot product, softmax and weighted average.

This is the simplest attention that works. Each token compares itself with every token using a dot product, softmaxes the scores, and mixes the token vectors. There are no learned matrices yet: those arrive in D3, and the scaling in D4. Each chapter adds one ingredient, so you can see what each one changes.

Three steps for the token "like"

WORKED EXAMPLE

X = [[1,0], [0,1], [1,1]] for I, like, tea. The query is like = [0, 1].

```
Step 1, scores:    like·I = 0,  like·like = 1,  like·tea = 1    → [0, 1, 1]
Step 2, softmax:   exp → [1, 2.718, 2.718], total 6.437        → [0.155, 0.422, 0.422]
Step 3, mix:       0.155·[1,0] + 0.422·[0,1] + 0.422·[1,1]      → [0.578, 0.845]
```

Step 1 is the middle row of the A1 table. Step 2 is A3. Step 3 is D1. There is nothing new here, only the combination.

Do the same for every token and you get one new vector per position:

Query	Scores	Weights (I, like, tea)	Output
I	[1, 0, 1]	[0.422, 0.155, 0.422]	[0.845, 0.578]
like	[0, 1, 1]	[0.155, 0.422, 0.422]	[0.578, 0.845]
tea	[1, 1, 2]	[0.212, 0.212, 0.576]	[0.788, 0.788]

Two weaknesses of this simple version

1. **Symmetric scores.** like·tea = tea·like, always. But relationships in language are often one-directional: a pronoun needs its noun more than the noun needs the pronoun.
2. **One vector does three jobs.** The same vector decides what a token looks for, what it advertises to others, and what it hands over when chosen. Tokens also tend to match themselves strongly, because $x \cdot x = |x|^2$.

Both weaknesses have the same fix: give each job its own learned projection. That is the next chapter.

CHAPTER SUMMARY

- Attention for one query: dot-product scores → softmax → weighted sum.
- Every token runs the same three steps, which produces a new vector per position.
- Using raw token vectors makes the scores symmetric and makes one vector do three jobs. D3 fixes both.

Exercises

D2.1 COMPUTE

Repeat the three steps for the query "I", and check your answer against the table.

D2.2 EXPLAIN

Why does "tea" put more weight on itself (0.576) than "like" puts on itself (0.422)?

Solutions are in Appendix B.

Next: queries, keys and values, three learned views of each token.

Why queries, keys and values are separate

Prerequisites: A2 Matrices, shapes and transpose, D2 Scores to weights, by hand · About 20 minutes

Goal: Explain what Q, K and V are, how they're computed, and what problem the separate projections solve.

In D2 each token's vector did three jobs at once. The Transformer splits them up: three learned matrices turn each token into three different vectors, one per job.

Three jobs, three vectors

Vector	Job	Search analogy
Query q	what this token is looking for	the search request you type
Key k	what this token offers for matching	a document's index terms
Value v	what this token hands over if chosen	the document's content

The scores become **query · key**, and the mixture is a weighted sum of **values**. In the model these are numeric vectors, not literal questions or documents. The analogy only describes the roles.

$$Q = X\,W_Q \qquad K = X\,W_K \qquad V = X\,W_V$$

X holds the token vectors, one per row. W_Q, W_K and W_V are learned parameter matrices, fixed after training. Q, K and V change with every input. In the base model each of these matrices is (512, 64) per head, so each token's query, key and value have 64 numbers (D5 explains why 64).

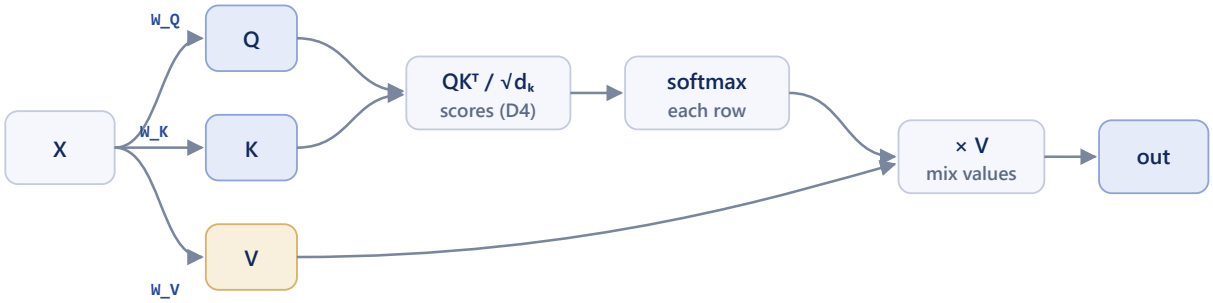


Figure D3.1. Queries and keys only decide the weights. Values are the content that actually flows to the output.

Separate matrices break the symmetry

The score table is $QK^T = X\,W_Q\,W_K^T\,X^T$. If W_Q = W_K, it is symmetric: A's score for B always equals B's score for A. With different matrices, it doesn't have to be. Try W_Q = [[1,1],[0,1]] (which maps [a, b] to [a, a + b]) and keep W_K as the identity:

```
Q = [[1,1], [0,1], [1,2]]     K = X = [[1,0], [0,1], [1,1]]

QKᵀ = [[1, 1, 2],     ← I's scores for I, like, tea
       [0, 1, 1],     ← like's scores
       [1, 2, 3]]     ← tea's scores
```

Now I → like scores 1, but like → I scores 0. The relationship has a direction. Training finds whatever directions turn out to be useful.

Separate values decouple "matching" from "content"

What makes a token easy to find can differ from what it should contribute once found. W_V lets the model pass along different features from the ones used for matching.

The running example from here on

To keep the arithmetic readable, the rest of Part D uses **W_Q = W_K = identity** and **W_V = 2 × identity**, so Q = K = X and V = 2X = [[2,0],[0,2],[2,2]]. Those are D1's values. Real projections are learned and far less tidy.

CHAPTER SUMMARY

- Q = XW_Q, K = XW_K, V = XW_V: three learned views of the same tokens.
- Query · key decides the weights. Values carry the content.
- Separate W_Q and W_K allow one-directional relationships, and a separate W_V decouples content from matching.

Exercises

D3.1 COMPUTE

X is (7, 512) and W_Q is (512, 64). What shape is Q? How many numbers does W_Q hold?

D3.2 EXPLAIN

Why is it called *self*-attention, and does "self" mean each position attends only to itself?

D3.3 EXPLAIN

During inference, which of W_Q, Q and V change from one sentence to the next?

Solutions are in Appendix B.

Next: put it all into one matrix equation, and add the $\sqrt{d_k}$ scale.

CHAPTER D4 · Part D · Attention, one piece at a time

The matrix form and the $\sqrt{d_k}$ scale

Prerequisites: A2 Matrices, shapes and transpose, A5 Mean, variance and normalization, D3 Queries, keys and values · About 25 minutes

Goal: Read and compute the paper's attention equation, $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$, and explain why the scale is there.

Everything so far fits in one line. This chapter computes it in full for the running example, one matrix at a time, and then explains its last ingredient, the division by $\sqrt{d_k}$.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^T / \sqrt{d_k} \right) V$$

Step by step, with Q = K = X and V = 2X

1 · EVERY SCORE AT ONCE: QK^T

$(3 \times 2) \mathbin{@} (2 \times 3) \to (3 \times 3)$. Row = query, column = key. This is the A1 table again.

```
QKᵀ = [[1, 0, 1],
       [0, 1, 1],
       [1, 1, 2]]
```

2 · SCALE: DIVIDE BY $\sqrt{D_k}$

Here $d_k = 2$ (the width of each query and key), so we divide by $\sqrt{2} \approx 1.414$.

```
QKᵀ/√2 ≈ [[0.707, 0,     0.707],
          [0,     0.707, 0.707],
          [0.707, 0.707, 1.414]]
```

3 · SOFTMAX EACH ROW SEPARATELY

Each row belongs to one query, and each row must sum to 1.

```
weights ≈ [[0.401, 0.198, 0.401],    ← I
           [0.198, 0.401, 0.401],    ← like
           [0.248, 0.248, 0.503]]    ← tea
```

4 · MIX THE VALUES: WEIGHTS @ V

(3 × 3) @ (3 × 2) → (3 × 2): one new vector per token.

```
output ≈ [[1.604, 1.198],    ← I
          [1.198, 1.604],    ← like
          [1.503, 1.503]]    ← tea
```

Compare "like" with D2: its weights moved from [0.155, 0.422, 0.422] to [0.198, 0.401, 0.401]. Scaling made them slightly more even, and doubling V doubled the size of what gets mixed.

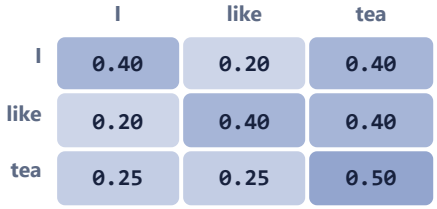


Figure D4.1. All attention weights for the running example (Q = K = X, scaled by √2). Rows are queries and columns are keys. Darker cells carry more weight.

Why divide by $\sqrt{d_k}$?

From A5: if the query and key entries are roughly random with variance 1, then $q \cdot k$ has variance d_k, and its typical size is $\sqrt{d_k}$. In the base model $d_k = 64$, so raw scores typically land around ±8. Softmax (A3, property 4) turns score gaps that large into nearly one-hot weights: one position takes almost everything.

That's bad for learning. When softmax is saturated, its gradients are tiny, so the model barely learns which positions should matter. The paper puts it directly: for large d_k, dot products grow large and push softmax into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ brings the typical score back to about 1, whatever the width.

Width d_k	Typical $\lvert q \cdot k \rvert$ ($\approx \sqrt{d_k}$)	Largest weight, unscaled	Largest weight, scaled by $\sqrt{d_k}$
2	≈ 1.4	40%	33%
16	≈ 4.0	75%	36%
64	≈ 8.0	88%	36%
512	≈ 22.6	96%	37%

Figure D4.2. One random query against 8 random keys (entries with variance 1), averaged over 2,000 draws. Without scaling, wider vectors push softmax towards putting nearly all the weight on one key. With scaling, the sharpness stays the same at every width.

Shapes for real sizes

Quantity	Shape (n tokens, one head)	Base model
Q, K	(n, d_k)	(n, 64)
QK^T, weights	(n, n)	(n, n)
V	(n, d_v)	(n, 64)
output	(n, d_v)	(n, 64)

The (n, n) table is where attention's n^2 cost comes from (C4).

CHAPTER SUMMARY

- $\text{softmax}(QK^T/\sqrt{d_k})V$: every score at once, scale, softmax each row, mix the values.
- Shapes: (n, d_k) @ (d_k, n) → (n, n) weights, then @ (n, d_v) → (n, d_v).
- $\sqrt{d_k}$ cancels the width-driven growth of dot products, which keeps softmax out of its flat, low-gradient region.

Exercises

D4.1 COMPUTE

Check the "tea" row: scores [0.707, 0.707, 1.414]. Exponentiate, normalize and mix V = [[2,0],[0,2],[2,2]].

D4.2 EXPLAIN

Softmax is applied to each *row* of QK^T, not to each column or to the whole table. Why?

Solutions are in Appendix B.

FURTHER READING

- Jay Alammar, The Illustrated Transformer. The self-attention section shows the same steps with pictures. `jalammar.github.io/illustrated-transformer/`

Next: run several attentions in parallel, which is multi-head attention.

Multi-head attention

Prerequisites: D4 Matrix form and the $\sqrt{d_k}$ scale · About 20 minutes

Goal: Understand how several attention heads run in parallel, how their outputs are combined, and why 8 heads of width 64 cost about the same as one head of width 512.

One attention head gives each query one set of weights, which is one way of deciding what matters. But a token may need several kinds of information at once: which noun a pronoun refers to, what the neighbouring words are, what the verb's object is. Multi-head attention gives the model several independent ways to look.

Each head has its own projections

Head i has its own W_Q^i, W_K^i and W_V^i. The same input therefore produces different queries, keys and values in each head, and so different attention patterns. Each head runs the D4 equation independently.

A learned pattern might help link a pronoun to its noun, while another tracks the previous word. These are interpretations researchers sometimes find, not jobs assigned by the programmer. Heads can also learn overlapping or hard-to-read behaviour.

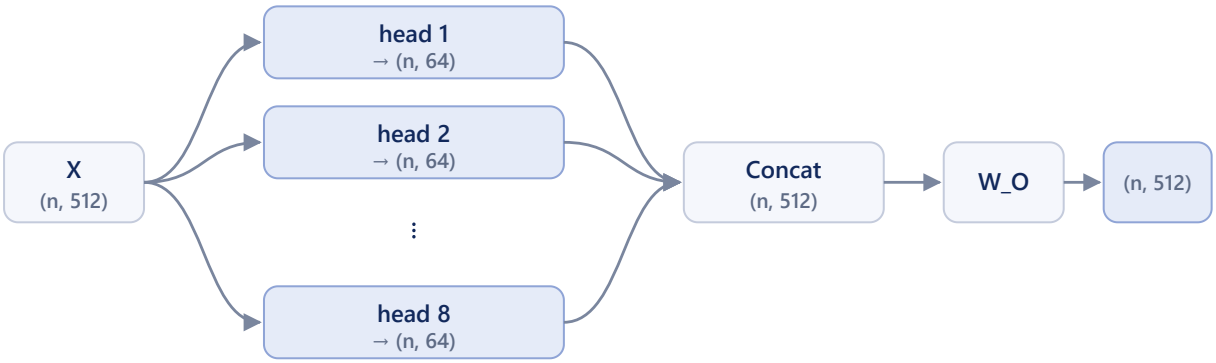


Figure D5.1. The base model uses 8 heads. Each head works in a 64-wide space, and 8 × 64 = 512, so the concatenation is back to full width.

```
head_i = Attention(X W_Q^i, X W_K^i, X W_V^i)
MultiHead(X) = Concat(head_1, …, head_h) W_O
```

Follow the shapes

Stage	Small example (3 tokens, width 8, 2 heads)	Base model (n tokens)
Input X	(3, 8)	(n, 512)
Each head's Q, K, V	(3, 4)	(n, 64)
Each head's weight table	(3, 3)	(n, n)
Each head's output	(3, 4)	(n, 64)
Concatenate the heads' features	(3, 8)	(n, 512)
Multiply by W_O	(3, 8)	(n, 512)

Concatenation places the heads' features side by side for each token. It doesn't add tokens. W_O then mixes information across heads.

Why 64? Splitting the width keeps the cost flat

The paper sets $d_k = d_v = d_model / h = 512 / 8 = 64$. Eight 64-wide heads have the same projection sizes as one 512-wide head: $8 \times (512 \times 64) = 512 \times 512$. So multi-head attention costs about the same as single-head attention, and buys eight distinct attention patterns per position. The paper found this worked better than one wide head. Too few or too many heads both hurt slightly.

In practice the heads aren't computed one after another. A single (512, 512) matrix produces all eight heads' queries at once, and the result is split into eight 64-wide slices that are processed side by side. The same happens for keys and values.

WHAT HEADS DON'T DO

Heads don't divide the sentence into groups of words. Each head attends over all allowed positions. What heads split is the *feature* dimension: each works in its own 64-wide projected space.

CHAPTER SUMMARY

- h heads run attention in parallel, each with its own projections.
- The outputs are concatenated, then mixed by W_O back to d_model.
- $d_k = d_model / h$ keeps the cost about equal to a single head.

Exercises

D5.1 COMPUTE

Three heads each output (6, 4). What shape do you get after concatenating them?

D5.2 COMPUTE

The big Transformer model uses d_model = 1024 and h = 16. What is d_k?

D5.3 EXPLAIN

What can 8 heads of width 64 do that 1 head of width 512 can't?

Solutions are in Appendix B.

Next: masks, which control what each position is allowed to see.

Masks: padding and causal

Prerequisites: D4 Matrix form and the $\sqrt{d_k}$ scale, A3 Exponentials, logs and softmax · About 25 minutes

Goal: Apply a padding mask and a causal mask to attention scores, and explain why the decoder needs the causal one.

Sometimes a position must *not* attend somewhere. It may be filler that only exists to make a batch rectangular. Or it may be a word the model is supposed to predict, which it must not be allowed to see. Both cases use the same trick, applied to the scores before softmax.

The trick: −∞ before softmax

Set a forbidden score to −∞. Since exp(−∞) = 0, that position gets weight exactly 0, and softmax renormalizes the remaining weights so they still sum to 1. (Zeroing weights *after* softmax would leave rows summing to less than 1.) In practice, implementations use a very large negative number such as -10^9 instead of a literal −∞.

Padding masks

Batches need equal-length sequences, so short sentences are filled with PAD tokens (C1). Take a batch of *Ich mag Tee* and *Hallo*, padded to "Hallo PAD PAD". A **padding mask** sets every score *toward* a PAD key to −∞, so no real token gathers information from filler. The outputs computed at PAD positions are simply ignored, and the loss skips them.

Padding masks are used everywhere: in encoder self-attention, in decoder self-attention, and on the source side of cross-attention.

Causal masks: no peeking at the answer

During training, the decoder receives the whole target sentence, shifted right by one position, and must predict the next token at every position:

```
Decoder inputs:  [START,  I,     like,  tea ]
Labels:          [I,      like,  tea,   END ]
```

The input at position "like" must predict "tea". If attention let it see the next input, which is "tea" itself, the task would be trivial copying, and the model would fail at generation time, when the future really is unknown. The **causal mask** lets position i attend only to positions ≤ i. That makes the allowed region of the score table a lower triangle.

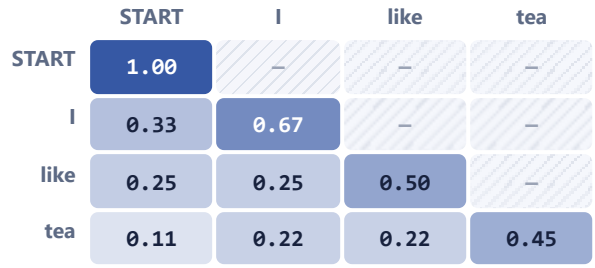


Figure D6.1. Causal-masked weights for the decoder inputs START, I, like, tea. Hidden cells (—) lie above the diagonal, and every row still sums to 1.

THE NUMBERS FOR "LIKE"

The scaled scores against START, I, like and tea are [0, 0, 0.707, 0.707].

```
no mask:   softmax([0, 0, 0.707, 0.707])  ≈ [0.165, 0.165, 0.335, 0.335]   ← sees "tea"
masked:    softmax([0, 0, 0.707, −∞])     ≈ [0.248, 0.248, 0.503, 0    ]   ← cannot
```

The weight that "tea" would have taken is redistributed across the allowed positions.

Masks are about access, not order

A causal mask restricts *which* positions can be seen. It doesn't tell the model *where* a token sits. Position information comes separately from positional encodings (D7). Don't treat them as the same operation.

Attention	Padding mask	Causal mask
Encoder self-attention	yes (source PAD)	no: the whole source is known
Decoder self-attention	yes (target PAD)	**yes**
Cross-attention (E3)	yes (source PAD)	no: the whole source may be consulted

CHAPTER SUMMARY

- Masking means setting forbidden scores to $-\infty$ before softmax, which gives those positions zero weight.
- Padding masks hide filler. Causal masks hide future positions in the decoder.
- The causal mask is what makes parallel training on whole target sentences honest.

Exercises

D6.1 COMPUTE

With the causal mask, what are the weights for the query "I"? Its scaled scores are [0, 0.707, 0, 0.707].

D6.2 EXPLAIN

Why can the decoder input "like" be used when predicting "tea", but the input "tea" cannot?

Solutions are in Appendix B.

Next: give attention a sense of word order.

Positional encoding

Prerequisites: C2 Embeddings, D4 Matrix form and the $\sqrt{d_k}$ scale · About 20 minutes

Goal: Understand why attention needs position information, and how the paper's sinusoidal encoding supplies it.

"Dog bites person" and "Person bites dog" contain the same words but describe very different events. Everything we've built so far would treat them identically.

Attention alone ignores order

Look at D4 again. Nothing in $\text{softmax}(QK^T/\sqrt{d_k})V$ refers to position. If you shuffle the rows of X, the output rows shuffle in exactly the same way, and each token gets the same vector as before. Without extra information, the model sees a *set* of tokens, not a sequence.

The fix: add a position vector to each embedding

The Transformer adds a **positional encoding** PE(pos) to each token's embedding. It has the same width (d_model), so the shape doesn't change. The same word at two positions now enters the network as two different vectors.

The paper's sinusoids

```
PE(pos, 2i)   = sin( pos / 10000^(2i / d_model) )
PE(pos, 2i+1) = cos( pos / 10000^(2i / d_model) )
```

Each pair of dimensions is a sine and cosine wave of a different frequency. The first pairs turn quickly, and later pairs turn more and more slowly. Think of a clock's second, minute and hour hands: the combined reading identifies a moment uniquely, and nearby moments have similar readings.

THE RUNNING EXAMPLE, WITH D_MODEL = 2

With only one pair (i = 0), the denominator is 1, so PE(pos) = [sin(pos), cos(pos)].

```
pos 0, I:     [1, 0] + [0.000,  1.000] = [1.000, 1.000]
pos 1, like:  [0, 1] + [0.841,  0.540] = [0.841, 1.540]
pos 2, tea:   [1, 1] + [0.909, -0.416] = [1.909, 0.584]
```

The real model first multiplies each embedding by $\sqrt{d_model}$, which we skip here to keep the numbers readable. Notice that "I" at position 0 now equals tea's raw embedding [1, 1]. With 2 dimensions, collisions like that are easy. With 512, they essentially never happen.

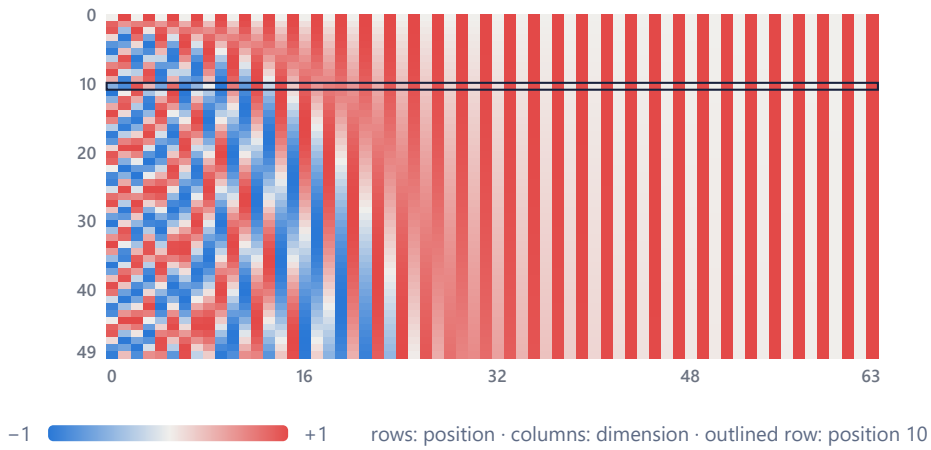


Figure D7.1. Sinusoidal positional encodings for positions 0–49 with d_model = 64. Left columns oscillate quickly and right columns slowly, so every row is a distinct code.

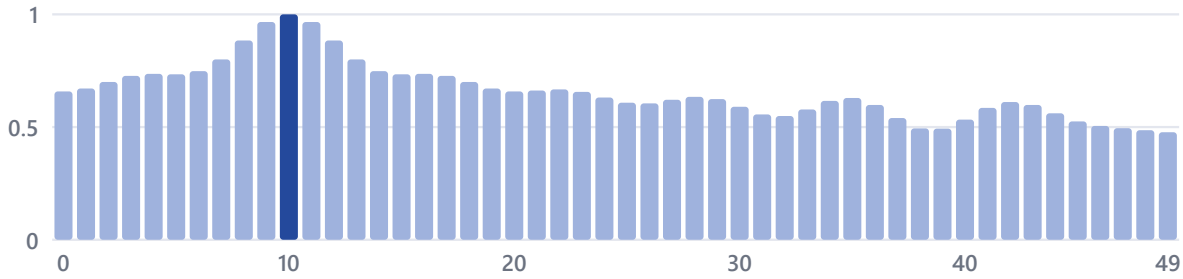


Figure D7.2. Similarity (cosine) between the encoding of position 10 and every other position. Neighbours score ≈ 0.97, while position 45 scores 0.53. Nearby positions have similar codes, which helps attention learn relative offsets.

Why sinusoids?

- **Relative positions are easy to use.** For any fixed offset k, PE(pos + k) is a linear function of PE(pos). It is a rotation of each sine/cosine pair. The paper hypothesized that this lets the model learn "attend 3 tokens back" easily.
- **No length limit is built in.** The formula works for any position, even ones longer than any training sentence.
- **Learned position vectors worked about as well.** The paper tried them and got nearly identical results, then kept sinusoids for the extrapolation argument.

Later models often use other schemes, such as learned positions, relative positions or rotary embeddings (G2). The *need* is the same, and you now know what it is.

CHAPTER SUMMARY

- Self-attention is blind to order. Positional encodings add that information.
- The paper adds sine/cosine waves of many frequencies, one value per dimension, to each embedding.
- Masking controls what can be seen. Positional encoding says where things are.

Exercises

D7.1 COMPUTE

Does adding a positional encoding double the embedding width?

D7.2 EXPLAIN

If you removed positional encodings, could the encoder tell "dog bites person" from "person bites dog"?

Solutions are in Appendix B.

Next: a checkpoint that asks you to compute attention from scratch.

Checkpoint D: attention from scratch

Prerequisites: D4 Matrix form and the $\sqrt{d_k}$ scale, D5 Multi-head attention, D6 Padding and causal masks, D7 Positional encoding · About 20 minutes

Goal: Prove to yourself that you can compute and explain attention without the chapter pages open.

Review questions

Q1 For 10 tokens and $d_k = 64$, what shape is the weight table of one head?

a. (10, 64)
b. (64, 64)
c. (10, 10)
d. (64, 10)

Q2 Why divide scores by $\sqrt{d_k}$?

a. Dot products grow with width; scaling keeps softmax from saturating, where gradients vanish
b. To make the weights sum to 1
c. To make the scores positive
d. To save memory

Q3 A causal mask is implemented by...

a. deleting future tokens from the input
b. zeroing future rows of V after softmax
c. adding positional encodings
d. setting future scores to $-\infty$ before softmax

Q4 8 heads each output (n, 64). After concatenation and W_O, the shape is...

a. (8, n, 64)
b. (n, 512)
c. (n, 64)
d. (8n, 64)

Q5 Without positional encodings, shuffling the input tokens of self-attention will...

a. change every output vector
b. cause an error
c. shuffle the outputs in the same way, each token keeping the same vector
d. leave the outputs in the original order

Q6 In Q = XW_Q, which part is learned and fixed after training?

a. W_Q
b. X
c. Q
d. all three

Answers and explanations are in Appendix C.

Exercises

D8.1 COMPUTE

Three tokens A = [1, 0], B = [0, 1], C = [2, 0]. Use Q = K = V = X, scaled by $\sqrt{2}$. Compute the attention output for query A. Write down the scores, scaled scores, weights and output.

D8.2 COMPUTE

Same tokens, with a causal mask. What are query B's weights?

D8.3 EXPLAIN

Explain multi-head self-attention to a colleague in under two minutes, without notes. Cover Q/K/V, the score table, the scale, softmax, mixing values, heads and masks.

Solutions are in Appendix B.

Part D is done. You can now compute attention. Part E assembles the whole Transformer around it.

PART E

The Transformer

The encoder, decoder and output layer assembled, then how the model is trained, how it generates, and how to read the paper.

E1	Feed-forward, residual connections and layer normalization
E2	The encoder block and stack
E3	The decoder: masked self-attention and cross-attention
E4	The output layer: from vectors to next-token probabilities
E5	One sentence through the whole model: a shape trace
E6	How the Transformer is trained
E7	How it generates: greedy decoding and beam search
E8	The whole architecture, and how to read the paper

Feed-forward, residual connections and layer normalization

Prerequisites: B1 Neurons and layers, A5 Mean, variance and normalization, D4 Matrix form and the $\sqrt{d_k}$ scale · About 20 minutes

Goal: Understand the three supporting components that turn attention into a block you can stack: what each one computes, and why it is there.

Attention moves information *between* positions. A Transformer block needs three more things: a way to process each token's gathered information, a way to keep signals and gradients flowing through many layers, and a way to keep numbers in a stable range. Those are the feed-forward network, residual connections and layer normalization.

1 · The position-wise feed-forward network

This is the same small two-layer network (B1) applied to each token's vector *separately*, with the same weights at every position:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(x W_1 + b_1) W_2 + b_2$$

In the base model W_1 is (512, 2048) and W_2 is (2048, 512): the network widens each token four times, applies ReLU, and narrows it back. A toy version with $2 \to 3 \to 2$:

WORKED EXAMPLE

```
x = [1, −1]     W₁ = [[1, 0,  1],      W₂ = [[1,   0  ],
                      [0, 1, −1]]            [0,   1  ],
                                             [0.5, 0.5]]   (biases 0)

x W₁ = [1, −1, 2]  →  ReLU  →  [1, 0, 2]  →  W₂  →  [1·1 + 2·0.5,  0 + 2·0.5] = [2, 1]
```

The division of labour is simple: **attention mixes across tokens, and the FFN transforms features within a token.** About two-thirds of an encoder block's parameters are in the FFN.

2 · Residual connections

Instead of replacing its input, each sublayer *adds* its output to the input: **x + Sublayer(x)**. For the running example's "like", the input [0, 1] plus the attention output from D4, [1.198, 1.604], gives **[1.198, 2.604]**.

Why it helps: each sublayer only has to learn a *correction* to what's already there, and there is always a direct path from the output back to the input. During backprop (B3), gradients flow along that path without being multiplied through every layer's weights, which makes deep stacks trainable. The shapes

must match, which is why every sublayer outputs d_model features.

3 · Layer normalization

Layer normalization normalizes *each token vector across its own features* (A5), then applies a learned scale γ and shift β:

```
LayerNorm(x) = γ · (x − mean(x)) / √(var(x) + ε) + β
```

WORKED EXAMPLE

x = [1, 2, 3, 6]: mean 3, variance 3.5, standard deviation ≈ 1.871.

```
(x − 3) / 1.871 ≈ [−1.069, −0.535, 0, 1.604]      (then × γ + β, which start at 1 and 0)
```

With only two features, the result is always [−1, 1], [1, −1] or [0, 0]. "like"'s [1.198, 2.604] becomes [−1, 1]. That is a quirk of tiny vectors, and one more reason real models use 512.

The ε is a tiny constant that prevents division by zero. Unlike softmax, LayerNorm's outputs can be negative and don't sum to anything in particular.

How they combine: Add & Norm

The paper wraps every sublayer the same way, with dropout (B4) applied to the sublayer's output:

```
output = LayerNorm( x + Dropout(Sublayer(x)) )
```

This is "post-LN": normalize after adding. Many later models use "pre-LN", x + Sublayer(LayerNorm(x)), which trains more stably in very deep stacks (G2). When reading other code, check which one it uses.

CHAPTER SUMMARY

- FFN: the same two-layer ReLU network applied to each token (512 → 2048 → 512).
- Residual: x + F(x), so sublayers learn corrections and gradients get a direct path back.
- LayerNorm normalizes each token's features. The paper applies LayerNorm(x + Sublayer(x)) around every sublayer.

Exercises

E1.1 COMPUTE

Normalize [2, 4] (with γ = 1 and β = 0). Then normalize [0, 0, 0, 4].

E1.2 COMPUTE

How many parameters does one base-model FFN have (weights and biases)?

E1.3 EXPLAIN

Which part directly mixes information across positions: attention, or the FFN? What would go wrong if a block had only FFNs?

Solutions are in Appendix B.

FURTHER READING

- He et al. (2015), Deep Residual Learning, where residual connections come from.
  `arxiv.org/abs/1512.03385`
- Ba et al. (2016), Layer Normalization.
  `arxiv.org/abs/1607.06450`

Next: assemble one encoder block, and stack six.

The encoder block and stack

Prerequisites: D5 Multi-head attention, D6 Padding and causal masks, D7 Positional encoding, E1 Feed-forward, residuals, layer norm · About 15 minutes

Goal: Draw one encoder block from memory, and explain what the six-block stack produces.

The encoder's job is to turn the source sentence into contextual vectors: one vector per source token that reflects the whole sentence. Every component is now familiar. This chapter puts them in the paper's order.

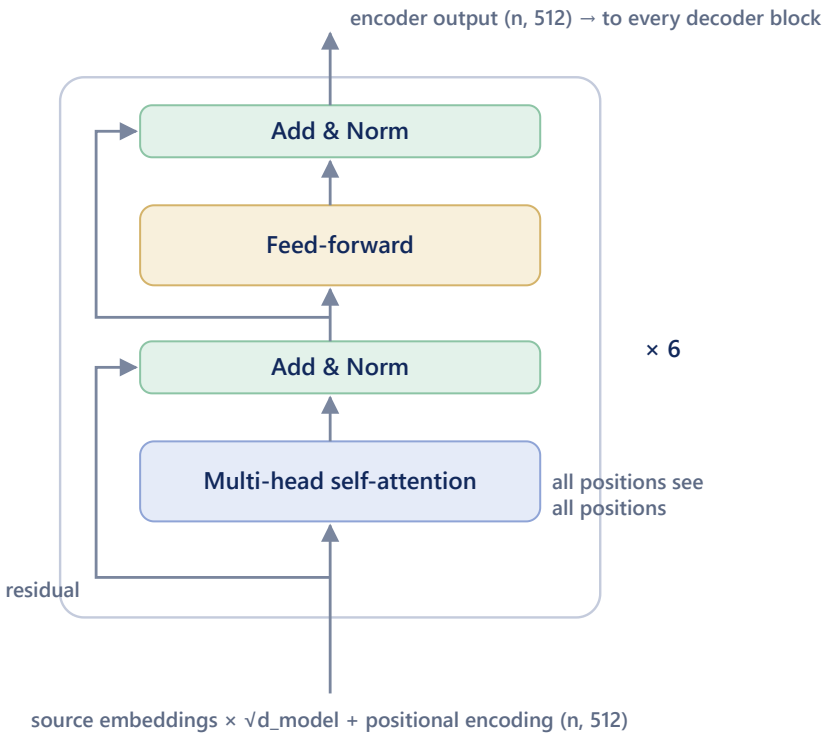


Figure E2.1. The grey paths that bend around a sublayer are residual connections. Their sum feeds Add & Norm.

The block, step by step

1. **Input:** source embeddings (C2), multiplied by √d_model, plus positional encodings (D7). Shape (n, 512).
2. **Multi-head self-attention** (D5) over the whole source. No causal mask, because the source is fully known. A padding mask applies if there are PAD tokens.
3. **Add & Norm** (E1).
4. **Feed-forward** (E1), applied to each position.
5. **Add & Norm** again.

Stacking: N = 6

The paper stacks six identical blocks, each with its own parameters. The shape stays (n, 512) all the way up, so blocks can be stacked freely, but the vectors grow more contextual with each block. Early blocks tend to capture local and surface patterns, later ones more abstract relationships. These are tendencies, not rules.

The final output is called the **memory**: one 512-number vector per source token. For *Ich mag Tee* that's a (3, 512) matrix. It is not a translation. It is a representation that the decoder will consult through cross-attention at every step.

The diagram omits dropout, which the paper applies to each sublayer's output and to the embedding sum during training. It's switched off at inference.

CHAPTER SUMMARY

- Encoder block = self-attention → Add & Norm → FFN → Add & Norm.
- Six blocks with separate weights. The shape stays (n, 512).
- The output "memory" is one contextual vector per source token, used by every decoder block.

Exercises

E2.1 COMPUTE

How many parameters does one base-model encoder block have? Use 4 × (512 × 512 + 512) for attention (W_Q, W_K, W_V and W_O with biases), 2,099,712 for the FFN, and 2 × 1,024 for the two LayerNorms.

E2.2 EXPLAIN

Draw the encoder block from memory: boxes, arrows and both residual paths. Then check it against the figure.

Solutions are in Appendix B.

Next: the decoder, with masked self-attention and cross-attention.

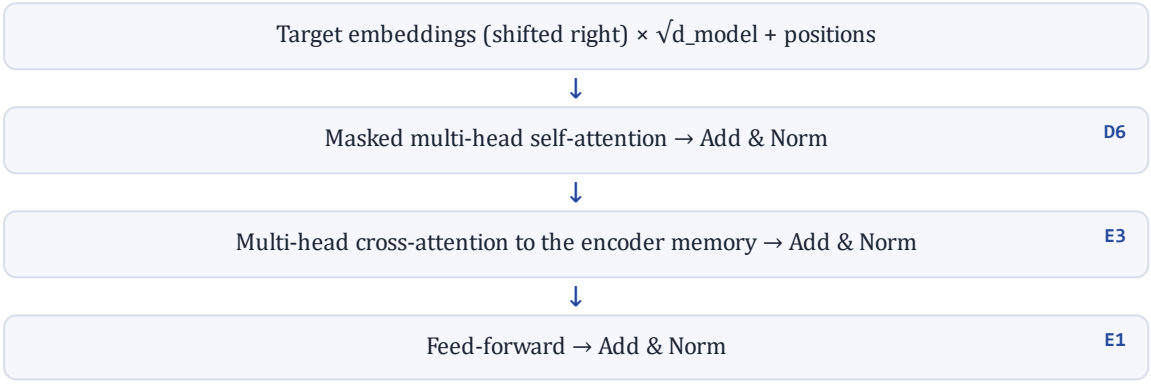
The decoder: masked self-attention and cross-attention

Prerequisites: D6 Padding and causal masks, E2 The encoder · About 25 minutes

Goal: Explain the decoder's three sublayers, and exactly where cross-attention gets its queries, keys and values.

The decoder has two jobs at every position: understand the target words produced so far, and decide which source words matter for the *next* word. It therefore needs two kinds of attention.

Three sublayers instead of two



Masked self-attention is D6: target positions attend to themselves and earlier target positions only. The **feed-forward** sublayer is E1. The new piece is the middle one.

Cross-attention

Cross-attention is the D4 equation with its inputs coming from two places:

	Comes from	Shape (4 target positions, 3 source tokens)
Queries Q	the decoder (output of its masked self-attention), × W_Q	(4, 64) per head
Keys K	the encoder memory, × W_K	(3, 64)
Values V	the encoder memory, × W_V	(3, 64)
Weights	softmax($QK^T/\sqrt{d_k}$)	**(4, 3)**: target rows × source columns
Output	weights @ V	(4, 64)

Each target position asks "given what I've produced so far, which source words do I need now?" The answer is a mixture of source information. This is the 2014 idea from C4, rebuilt with Transformer attention. There is no causal mask here: the whole source may always be consulted. Only source padding is masked.

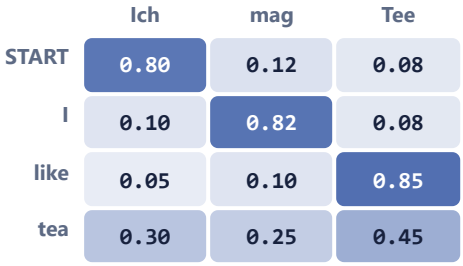


Figure E3.1. Illustrative cross-attention weights, like those a trained model might produce. Rows are decoder inputs and columns are source tokens.

Look at the pattern. The row for input "like" focuses on *Tee*, because position "like" is predicting the *next* word, "tea". Each row looks at the source word needed for the following target word. The weights are invented for illustration. Real heads are messier, and different heads attend differently.

Six decoder blocks, one shared memory

The paper stacks six decoder blocks. Every one of them cross-attends to the *same* final encoder output, not to the encoder block at the same depth. After the sixth block, a linear layer and softmax turn each position's vector into next-token probabilities (E4).

Training versus generation

In training, the whole shifted target is known, so all four positions are computed in parallel and the causal mask keeps the computation honest. In generation, the model has only the tokens it has produced so far: it predicts one, appends it, and runs again (E7).

CHAPTER SUMMARY

- Decoder block = masked self-attention → cross-attention → FFN, each with Add & Norm.
- Cross-attention: Q from the decoder, K and V from the encoder memory. Weights are (target × source).
- All six decoder blocks consult the same final encoder output.

Exercises

E3.1 COMPUTE

The target has 9 positions and the source has 12 tokens. What shape is one head's cross-attention weight table? And the decoder's self-attention table?

E3.2 EXPLAIN

Explain the difference between encoder self-attention, decoder masked self-attention and cross-attention.

E3.3 EXPLAIN

Why does cross-attention need no causal mask?

Solutions are in Appendix B.

Next: turn the decoder's final vectors into word probabilities.

The output layer: from vectors to next-token probabilities

Prerequisites: A3 Exponentials, logs and softmax, B2 Loss functions, C2 Embeddings · About 15 minutes

Goal: Compute next-token probabilities from a decoder vector, and understand the paper's weight sharing.

After six decoder blocks, each target position has a 512-number vector. A prediction needs one probability for each of the ~37,000 vocabulary tokens. Two familiar operations bridge the gap: a linear layer and softmax.

Linear → logits → softmax

```
logits = h W_out + b          (512) @ (512, 37000) → (37000) scores
probabilities = softmax(logits)
```

This is the classifier from B2, with one class per vocabulary token. It is applied at every target position.

Weight tying: reuse the embedding table

The paper **shares one matrix** between the source embedding, the target embedding and this output layer. Concretely, $W_{out} = E^T$. So the logit for token j is the dot product (A1) between the decoder's vector h and token j's embedding. The model scores each candidate word by how well h aligns with that word's vector. Sharing also saves about 19 million parameters per copy. (The shared English–German BPE vocabulary from C1 is what makes this possible.)

Because the same matrix is used both ways, the paper multiplies embeddings by $\sqrt{d_{model}}$ on the input side, so that the embeddings stay in proportion to the positional encodings added to them.

WORKED EXAMPLE WITH A 5-TOKEN VOCABULARY

Embedding rows: END = [−1, −1], I = [1, 0], like = [0, 1], tea = [1, 1], coffee = [1, 0.8]. The final decoder vector at position "like" is h = [1.2, 1.1].

```
logits = h · each row = [−2.30, 1.20, 1.10, 2.30, 2.08]
softmax               ≈ [0.004, 0.136, 0.123, 0.409, 0.328]    ← tea most likely
```

"coffee" comes a close second, because its embedding is close to tea's (C2). If the label is "tea", the loss is $-\ln 0.409 \approx$ **0.895** (B2).

CHAPTER SUMMARY

- Final vector → linear layer → logits over the vocabulary → softmax → probabilities.
- The paper ties the output matrix to the embedding table, so logit = h · word embedding.
- Training minimizes −ln(probability of the correct next token) at every position.

Exercises

E4.1 COMPUTE

With the same embeddings, h = [−1, −1.2]. Which token gets the highest logit?

E4.2 EXPLAIN

With weight tying, why do words with similar embeddings get similar probabilities?

Solutions are in Appendix B.

Next: follow one sentence through every layer, with real shapes.

One sentence through the whole model: a shape trace

Prerequisites: D5 Multi-head attention, E2 The encoder, E3 The decoder and cross-attention, E4 The output layer · About 20 minutes

Goal: Track the exact tensor shape at every stage of the base model, and estimate its parameter count yourself.

Shapes are the fastest way to check your understanding. If you can say what shape comes out of every box, you understand the box. Here is *Ich mag Tee → I like tea* through the base model, as one training example with batch size 1.

#	Stage	Shape	Note
1	Source token IDs: Ich, mag, Tee	(3)	C1
2	Source embeddings × √512	(3, 512)	row lookup, C2
3	+ positional encoding	(3, 512)	addition, D7
4	Per head: Q, K, V = X W	(3, 64) each, × 8 heads	D3, D5
5	Per head: QK^T / √64, softmax	(3, 3)	D4
6	Per head: weights @ V	(3, 64)	D4
7	Concatenate 8 heads, × W_O	(3, 512)	D5
8	Add & Norm, FFN (512 → 2048 → 512), Add & Norm	(3, 512)	inner (3, 2048), E1
9	× 6 encoder blocks → memory	(3, 512)	E2
10	Target input IDs: START, I, like, tea	(4)	shifted right, D6
11	Target embeddings × √512 + PE	(4, 512)	same shared table
12	Masked self-attention, per head weights	(4, 4)	lower triangle
13	Cross-attention, per head weights	(4, 3)	Q from target, K/V from memory
14	FFN, Add & Norm, × 6 decoder blocks	(4, 512)	E3
15	Linear (tied to embeddings)	(4, 37000)	logits, E4
16	Softmax over the last dimension	(4, 37000)	each row sums to 1
17	Cross-entropy against labels I, like, tea, END	(4) → mean → scalar	B2, E6

With a batch of B sentence pairs, put B in front of every shape: (B, 3, 512), (B, 8, 3, 3) and so on. Nothing else changes.

Count the parameters yourself

Component	Calculation	Parameters
Shared embedding / output matrix	37,000 × 512	18,944,000
One attention sublayer	4 × (512 × 512 + 512)	1,050,624
One FFN	512 × 2048 + 2048 + 2048 × 512 + 512	2,099,712
One LayerNorm	γ and β, 2 × 512	1,024
Encoder block	1 attention + 1 FFN + 2 LayerNorms	3,152,384
Decoder block	2 attention + 1 FFN + 3 LayerNorms	4,204,032
Total	embedding + 6 × encoder + 6 × decoder	**≈ 63.1 million**

The paper reports 65 million for the base model. The small gap comes from details such as the exact vocabulary size. Notice where the parameters live: in the FFNs and the embedding table, not in attention. Attention's cost is mostly in *computation* (the n × n table), not in parameters.

CHAPTER SUMMARY

- The width stays 512 between sublayers. Only the FFN's inside (2048) and the logits (vocabulary) differ.
- Attention tables are (n, n), or (target, source) for cross-attention. That is where the n^2 cost comes from.
- About 63M parameters by hand, close to the paper's 65M, mostly in FFNs and embeddings.

Exercises

E5.1 COMPUTE

A batch of 32 sentence pairs, source padded to 20 tokens and target to 22. Give the shapes of: the encoder output, one decoder self-attention weight table for all heads, and the logits.

E5.2 EXPLAIN

Which stage's shape depends on vocabulary size, and which on sentence length squared?

Solutions are in Appendix B.

Next: how the model is trained.

How the Transformer is trained

Prerequisites: B4 Training in practice, D6 Padding and causal masks, E4 The output layer · About 25 minutes

Goal: Describe one training step of the paper's model, including teacher forcing, the learning-rate schedule and label smoothing.

Part B covered general training. This chapter covers the specific recipe in the paper's Section 5, which the first-reading guide in E8 tells you to leave for later. You now have everything needed to read it.

Teacher forcing: train every position at once

The decoder receives the *correct* target, shifted right ([START, I, like, tea]), rather than its own guesses. This is **teacher forcing**. Combined with the causal mask (D6), every target position is a separate next-token prediction, and all of them are computed in one parallel pass. A 30-token sentence gives 30 training signals in one step. This is a big part of why Transformers train so much faster than RNNs.

One consequence: in training, the model always sees a perfect prefix. At generation time it sees its own possibly imperfect output. The mismatch is called *exposure bias*. It's usually tolerable, but worth knowing about.

The loss

Cross-entropy (B2) at every non-PAD target position, averaged. Padding positions are excluded through the same mask idea as D6.

The learning-rate schedule

```
lrate = d_model^(−0.5) · min( step^(−0.5),  step · warmup_steps^(−1.5) )
```

For the first warmup_steps (4,000), the second term is smaller, so the rate grows linearly. After that, the first term is smaller, so the rate decays in proportion to $1/\sqrt{\text{step}}$. The peak is at step 4,000: 512^(−0.5) × 4000^(−0.5) ≈ **0.0007**.

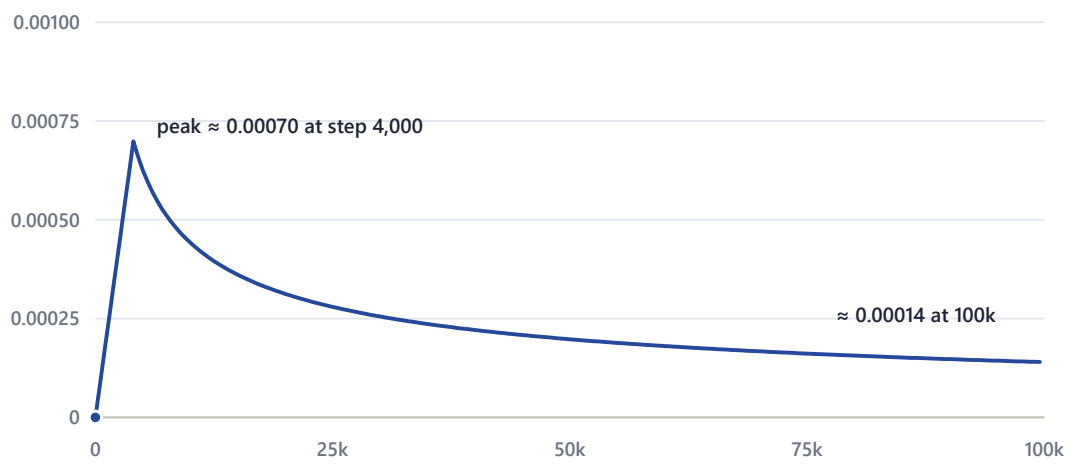


Figure E6.1. The paper's learning-rate schedule (d_model = 512, warmup_steps = 4,000): a linear rise, then decay proportional to 1/√step. The horizontal axis is training steps.

Why warm up? At the start, the weights are random and Adam's running averages (B4) are still empty, so early gradients are unreliable. Large steps then can push a post-LN Transformer into a bad region from which it never recovers. Starting small and ramping up avoids that.

Label smoothing

Instead of a hard target (100% on the right token), the paper trains against a softened target with ε_ls = 0.1. In the common formulation, the target keeps $1 - \varepsilon$ of its mass on the correct token and spreads ε evenly across the vocabulary:

```
vocabulary of 4, correct = token 2:   hard [0, 0, 1, 0]   →   smoothed [0.025, 0.025, 0.925, 0.025]
```

This discourages the model from becoming extremely confident. The paper notes it *hurts perplexity*, because the model learns to be more unsure, but *improves accuracy and BLEU*.

The rest of the recipe

Ingredient	Base model
Data	WMT 2014 English–German, about 4.5M sentence pairs (and 36M for English–French)
Batching	sentences grouped by length, about 25,000 source + 25,000 target tokens per batch
Optimizer	Adam, $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\varepsilon = 10^{-9}$, with the schedule above
Dropout	0.1 on every sublayer output and on the embedding + PE sums
Label smoothing	$\varepsilon_{ls} = 0.1$
Hardware and time	8 NVIDIA P100 GPUs, 100,000 steps, about 12 hours (the big model: 300,000 steps, 3.5 days)
Final model	an average of the last 5 checkpoints

CHAPTER SUMMARY

- Teacher forcing plus the causal mask gives one parallel pass that trains every target position.
- The learning rate warms up linearly for 4,000 steps, then decays like $1/\sqrt{\text{step}}$.
- Label smoothing (0.1) trades a little perplexity for better translations.

Exercises

E6.1 COMPUTE

Using the schedule formula, what is the learning rate at step 1,000 and at step 16,000 (warmup 4,000)?

E6.2 COMPUTE

Label-smooth a vocabulary of 10 with $\varepsilon = 0.1$, where the correct token is index 3.

E6.3 EXPLAIN

Why does teacher forcing plus a causal mask let the decoder train in parallel, when generation must be sequential?

Solutions are in Appendix B.

FURTHER READING

- Szegedy et al. (2015), where label smoothing was introduced.
  `arxiv.org/abs/1512.00567`

Next: how a trained model actually produces a translation.

How it generates: greedy decoding and beam search

Prerequisites: E4 The output layer, E3 The decoder and cross-attention · About 20 minutes

Goal: Run the generation loop by hand, and show with an example why beam search can beat greedy decoding.

A trained model outputs a probability for every possible next token. To translate, we still have to *choose* tokens, one at a time, feeding each choice back in. The way we choose matters.

The autoregressive loop

1. Run the encoder **once** on *Ich mag Tee* to get the memory.
2. Start the decoder with [START].
3. Run the decoder, take the last position's probabilities, and choose a token.
4. Append it: [START, I]. Repeat step 3.
5. Stop when END is chosen, or at a maximum length. The paper uses input length + 50.

This is why generation is sequential even though training is parallel: token t+1 can't be chosen until token t exists.

Greedy decoding and its trap

Greedy decoding picks the single most likely token at every step. It's fast, but a locally best choice can lead to a worse sentence overall.

WORKED EXAMPLE (MADE-UP PROBABILITIES)

Step 1	P	Step 2, given step 1	Sequence probability
The	0.5	dog 0.4 · cat 0.3 · …	The dog = 0.5 × 0.4 = **0.20**
A	0.4	dog 0.9 · …	A dog = 0.4 × 0.9 = **0.36**
My	0.1	…	—

Greedy takes "The" (0.5 > 0.4) and ends with "The dog" at 0.20. But "A dog" has probability 0.36. Greedy never looked down that path.

Beam search

Beam search keeps the k best partial sequences (the "beam") at every step. It extends each one by every possible token and keeps the k best of the results. With k = 2 in the example: keep "The" (0.5) and "A" (0.4), expand both, and find "A dog" = 0.36 as the winner. In practice, implementations add log-probabilities rather than multiplying probabilities, to avoid tiny numbers.

Products of probabilities shrink with every token, so raw beam search favours short outputs. A **length penalty** normalizes for that. The paper used beam size **4** and length penalty α = **0.6**.

Sampling and temperature (modern chat models)

Translation wants the most likely output. Open-ended text often wants variety. **Sampling** draws the next token at random according to the probabilities. **Temperature** divides the logits before softmax. That is exactly the scale effect from A3: below 1 is sharper and more predictable, above 1 is flatter and more varied. Top-k and top-p sampling restrict the draw to the most likely tokens.

A note on cost

Done naively, each new token re-runs the decoder over the whole prefix. But the keys and values of earlier positions never change, so they can be stored and reused. This **KV cache** (G2) is standard in every modern system.

CHAPTER SUMMARY

- Generation is a loop: predict, choose, append, repeat until END.
- Greedy decoding is fast but short-sighted. Beam search keeps the k best paths (the paper uses k = 4, α = 0.6).
- Temperature is the softmax scale from A3, applied to sampling.

Exercises

E7.1 COMPUTE

In the example, what would beam search with k = 1 return?

E7.2 EXPLAIN

Why does the encoder run once per translation, while the decoder runs once per output token?

Solutions are in Appendix B.

Next: you're ready to read the paper itself.

The whole architecture, and how to read the paper

Prerequisites: E2 The encoder, E3 The decoder and cross-attention, E4 The output layer, E6 How it's trained · About 30 minutes

Goal: Trace the model end to end, then read *Attention Is All You Need* in three focused passes, knowing which chapter covers each section.

Go back to the diagram in chapter 0.1. Every box should now be something you can explain and compute. This chapter ties it together and turns you loose on the original paper.



Trace one prediction

The encoder processes *Ich mag Tee* once. The decoder processes a prefix such as [START, I]. Its masked self-attention combines the prefix. Cross-attention pulls in source information, mostly from *mag*. The FFN transforms the result, and six blocks repeat this. The last position's vector is scored against every word's embedding, softmax gives probabilities, and "like" comes out on top. During training, cross-entropy against the true next token sends gradients back through every one of those steps.

Read the paper in three passes

Pass	Read	Question to answer	Chapters
1 · Purpose	Abstract, §1, §2, Figure 1, §7	What changes compared with recurrent models, and why?	C3, C4, 0.1
2 · Mechanism	§3.1, then §3.4–3.5, then §3.2 (all subsections), then §3.3	Can I explain each box and compute one attention output?	D1–D7, E1–E4
3 · Evidence	§4, §5, §6 (including Table 3, the model variations)	What's the efficiency argument, how was the model trained, and what did each design choice contribute?	E5–E7

In pass 2, §3.2.1 contains the $\sqrt{d_k}$ argument (D4), §3.2.2 the 512/8 = 64 split (D5), and §3.2.3 the three uses of attention: encoder self, decoder masked self, and encoder–decoder (E3).

§4 in one table: why self-attention?

Layer type	Compute per layer	Sequential steps	Longest path between two positions
Self-attention	$O(n^2 \cdot d)$	O(1)	O(1)
Recurrent	$O(n \cdot d^2)$	O(n)	O(n)
Convolutional (kernel k)	$O(k \cdot n \cdot d^2)$	O(1)	O(log_k n)

The rows connect directly to C3's three problems. Self-attention has constant path length (no long chains) and constant sequential steps (full parallelism). When n is smaller than d, as it is for sentences ($n \approx 30$, $d = 512$), it's even cheaper per layer than recurrence. The n^2 term only dominates for long sequences.

Base versus big

Symbol	Meaning	Base	Big
N	blocks per stack	6	6
d_model	token width	512	1024
d_ff	FFN inner width	2048	4096
h	heads	8	16
$d_k = d_v$	per-head width	64	64
P_drop	dropout	0.1	0.3
—	parameters	65M	213M

The big model reached 28.4 BLEU on English→German, beating the previous best results, including ensembles, at a fraction of their training cost. BLEU measures overlap with reference translations. It's a useful benchmark, but not a complete measure of translation quality.

The paper: arxiv.org/abs/1706.03762. A visual companion: jalammar.github.io/illustrated-transformer.

CHAPTER SUMMARY

- You can now explain every box in Figure 1 and compute what it does.
- Read the paper in three passes: purpose, mechanism, evidence.
- §4's table is C3's three problems, solved.

Exercises

E8.1 EXPLAIN

Without notes: walk through Figure 1 of the paper box by box, giving the shape coming out of each box for a 3-token source and a 4-token target.

E8.2 EXPLAIN

Name three things the title "Attention Is All You Need" does *not* mean.

Solutions are in Appendix B.

Part E is done. Finally, Part G: how this design became today's models.

PART G

After the paper

How the 2017 design became BERT, GPT and today's large language models.

G1	Encoder-only, decoder-only and encoder–decoder
G2	What modern models changed, and what they kept
G3	Pretraining, fine-tuning and instruction tuning

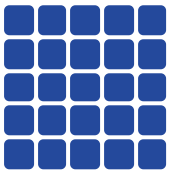
Encoder-only, decoder-only and encoder–decoder

Prerequisites: E2 The encoder, E3 The decoder and cross-attention, D6 Padding and causal masks · About 15 minutes

Goal: Place BERT, GPT and T5 relative to the 2017 design, and know which family suits which job.

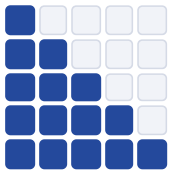
Within two years of the paper, researchers found that you don't always need both halves. Keep only the encoder, keep only the decoder, or keep both. Each choice changes the attention mask and the training objective, and so what the model is good at.

Encoder-only



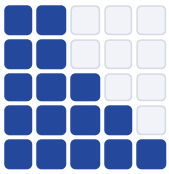
Every token sees every token (bidirectional).

Decoder-only



Each token sees only earlier tokens (causal, D6).

Encoder–decoder



Bidirectional over the input; causal over the output, plus cross-attention.

Each grid shows which positions (columns) a position (row) may attend to. Filled cells are allowed.

Encoder-only: BERT (2018)

BERT keeps just the encoder stack. Because every token sees both left and right context, it can't be trained on "predict the next word", which would be trivial when the next word is visible. Instead it uses **masked language modelling**: hide about 15% of the input tokens and predict them from the surrounding context. BERT-style models excel at *understanding* tasks such as classification, named-entity tagging and producing embeddings for search. They don't generate long text naturally.

Decoder-only: GPT (2018 onward)

GPT keeps the decoder stack and removes cross-attention, since there is no encoder to consult. Each block is masked self-attention followed by the FFN. Training is plain next-token prediction on huge amounts of text, using exactly the teacher forcing and causal mask from E6. The prompt and the response are simply one sequence: the model reads the prompt and keeps predicting the next token (E7). Almost all of today's chat assistants are decoder-only.

Encoder–decoder: the original, and T5 (2019)

T5 kept the full 2017 structure and framed every task as text-to-text: "translate: …", "summarize: …", "question: …". Encoder–decoder models remain a natural fit when there is a distinct input to transform, as in translation or summarization.

Family	Stacks used	Attention mask	Training objective	Best at	Examples
Encoder-only	encoder	bidirectional	predict masked tokens	classification, retrieval, embeddings	BERT, RoBERTa
Decoder-only	decoder without cross-attention	causal	predict the next token	generation, chat, general-purpose use	GPT series, most LLMs
Encoder–decoder	both	bidirectional + causal + cross	predict the target sequence	translation, summarization	original Transformer, T5, BART

CHAPTER SUMMARY

- Encoder-only (BERT): bidirectional, trained on masked tokens, good at understanding tasks.
- Decoder-only (GPT): causal, trained on next-token prediction, good at generation. This is the basis of modern LLMs.
- Encoder–decoder (original, T5): a natural fit for input → output transformations.

Exercises

G1.1 EXPLAIN

Why can't BERT be trained with next-token prediction?

G1.2 EXPLAIN

A decoder-only model translates "Ich mag Tee" when prompted with "German: Ich mag Tee. English:". Where did the encoder and cross-attention go?

Solutions are in Appendix B.

FURTHER READING

- Devlin et al. (2018), BERT.
  `arxiv.org/abs/1810.04805`
- Brown et al. (2020), GPT-3: Language Models are Few-Shot Learners.
  `arxiv.org/abs/2005.14165`
- Raffel et al. (2019), T5.
  `arxiv.org/abs/1910.10683`

Next: what has changed inside the blocks since 2017.

What modern models changed, and what they kept

Prerequisites: E1 Feed-forward, residuals, layer norm, D7 Positional encoding, E7 How it generates text · About 20 minutes

Goal: Recognize the common refinements in today's Transformers, and connect each one to the original component it modifies.

Open the description of a current large language model and you'll find the 2017 skeleton almost intact: embeddings, stacked blocks of attention plus FFN, residual connections, normalization, softmax over a vocabulary. The changes are refinements of individual boxes. Because you know what each box does, each change is easy to follow.

Component	2017 paper	Common today	Why	Builds on
Normalization placement	post-LN: LN(x + F(x))	pre-LN: x + F(LN(x))	A clean residual path makes very deep stacks train stably	E1
Normalization type	LayerNorm	often RMSNorm	Skips the mean subtraction: cheaper, and works as well	A5, E1
FFN activation	ReLU	GELU, or gated variants such as SwiGLU	Smoother or gated activations improve quality at the same size	B1, E1
Positions	sinusoids added to embeddings	rotary embeddings (RoPE), or ALiBi biases	Encode *relative* offsets directly inside q·k	D7
Generation	recompute the prefix each step	KV cache	Store past keys and values instead of recomputing them	E7
Heads for K, V	separate K and V per head	shared across groups of heads (MQA, GQA)	Shrinks the KV cache, so generation is faster	D5
Attention computation	full n × n matrix in memory	FlashAttention	Same exact result, computed in tiles. Much less memory traffic	D4
FFN capacity	one FFN per block	sometimes mixture-of-experts	Many expert FFNs, a router picks a few per token: more parameters, similar compute	E1
Scale	65M–213M parameters	billions to hundreds of billions	Quality improved predictably with more data, parameters and compute	E5

Four changes worth understanding properly

Pre-LN. In the original, every residual sum passes through a LayerNorm, so the "highway" from output to input is interrupted at every block. Pre-LN normalizes only the sublayer's *input* and leaves the residual sum untouched. Gradients get an uninterrupted path, deep models train reliably, and the strict need for warmup (E6) is reduced.

Rotary position embeddings (RoPE). In D7 you saw that a fixed offset corresponds to a *rotation* of each sine/cosine pair. RoPE takes that idea literally. Instead of adding a position vector, it rotates each query and key pair of dimensions by an angle proportional to the token's position. The dot product q·k then depends on the *difference* between positions, which is exactly the relative information attention needs.

The KV cache. During generation (E7), the keys and values of earlier tokens never change, because the causal mask stops later tokens from affecting them. So they are computed once and stored. Each new token computes only its own query, key and value, then attends over the cache. Generation becomes far cheaper per token, at the cost of memory that grows with context length. That is the motivation for sharing K and V across heads (GQA).

The n^2 problem. The (n × n) score table (D4) dominates for long inputs: doubling the context quadruples the attention work. FlashAttention makes the exact computation much faster in practice. Other approaches, such as sliding-window or sparse attention, restrict which pairs are scored.

CHAPTER SUMMARY

- Modern LLMs keep the 2017 skeleton and refine individual components.
- The most common changes: pre-LN, RMSNorm, gated FFNs, rotary positions, the KV cache, shared K/V heads, FlashAttention.
- Every refinement is easy to understand once you know the component it modifies.

Exercises

G2.1 EXPLAIN

Why is the KV cache valid in a decoder-only model, that is, why don't earlier tokens' keys and values change when new tokens arrive?

G2.2 EXPLAIN

Which original component does each of these modify: SwiGLU, RoPE, GQA, RMSNorm?

Solutions are in Appendix B.

FURTHER READING

- Xiong et al. (2020), On Layer Normalization in the Transformer Architecture (pre-LN versus post-LN).
  `arxiv.org/abs/2002.04745`
- Su et al. (2021), RoFormer: rotary position embeddings.
  `arxiv.org/abs/2104.09864`
- Dao et al. (2022), FlashAttention.
  `arxiv.org/abs/2205.14135`
- Ainslie et al. (2023), GQA.
  `arxiv.org/abs/2305.13245`
- Shazeer (2020), GLU Variants Improve Transformer.
  `arxiv.org/abs/2002.05202`

Next: how such models are trained to be useful assistants.

Pretraining, fine-tuning and instruction tuning

Prerequisites: G1 BERT, GPT and T5, E6 How it's trained · About 15 minutes

Goal: Understand the stages that turn a Transformer into a useful model, and close the loop on everything this book covered.

The 2017 model was trained from scratch for one task, translation, on labelled sentence pairs. Modern practice splits training into stages. A general model is pretrained once on enormous amounts of unlabelled text, then adapted cheaply to many uses.

Stage 1 · Pretraining

Train on a vast text corpus with a **self-supervised** objective: next-token prediction for decoder-only models, or masked tokens for encoder-only ones (G1). The labels come for free, because the text itself is the answer key. Through this one objective, the model absorbs grammar, facts and patterns of reasoning, because predicting the next word well requires them. The mechanics are exactly E6: teacher forcing, the causal mask and cross-entropy, at vastly larger scale.

Stage 2 · Fine-tuning

Continue training the pretrained model on a much smaller, task-specific labelled dataset, such as sentiment labels or medical question–answer pairs. Starting from pretrained weights needs far less data than training from scratch.

Stage 3 · Instruction tuning and preference tuning

A model pretrained only on next-token prediction continues text. It doesn't necessarily follow requests. **Instruction tuning** fine-tunes it on examples of instructions paired with good responses. **Preference tuning** then teaches it which of two responses people prefer. One method is RLHF, reinforcement learning from human feedback, which trains a reward model on human comparisons and then optimizes against it. Another is DPO, direct preference optimization, which learns from the comparison pairs directly.

In-context learning: adapting without training

Large pretrained models can often perform a new task just from instructions or a few examples written in the prompt, with *no parameter update at all*. This fits B3's distinction between training and inference: the parameters stay fixed, and only the input changes. All the adaptation happens through attention over the prompt.

Stage	Data	Objective	Changes parameters?
Pretraining	huge unlabelled text	next / masked token	yes: all of them, from scratch
Fine-tuning	small labelled task data	task loss	yes, starting from pretrained weights
Instruction / preference tuning	instruction–response pairs, human preferences	imitate good responses, prefer better ones	yes
In-context learning	examples written in the prompt	none: ordinary inference	no

What you now understand

You started with dot products and softmax. You can now explain how a sentence becomes vectors, how attention mixes them, how masks and positions shape that mixing, how the encoder and decoder are assembled, how the model is trained and how it generates text. You can also see how those same pieces became BERT, GPT and today's large language models. Retake the checkpoints (B5, C5, D8) in a week. Retrieval spaced out over time is what makes knowledge last.

CHAPTER SUMMARY

- Pretraining is self-supervised and learns general ability from raw text. Fine-tuning and instruction tuning adapt that ability cheaply.
- Preference tuning (RLHF, DPO) aligns outputs with what people find helpful.
- In-context learning adapts behaviour through attention over the prompt, with no weight updates.

Exercises

G3.1 **EXPLAIN**

Explain to a non-technical friend, in five sentences, how a chatbot produces an answer. Use the words token, attention, probability and "one at a time".

G3.2 **EXPLAIN**

Which chapter's idea makes in-context learning possible without changing any weights?

Solutions are in Appendix B.

FURTHER READING

- Ouyang et al. (2022), Training language models to follow instructions with human feedback (InstructGPT). `arxiv.org/abs/2203.02155`
- 3Blue1Brown, the Transformer and attention chapters, a visual review of everything in Part D. `www.3blue1brown.com/topics/neural-networks`

You've reached the end of the book. Now open the paper and read it in three passes (E8). Every section of it should feel familiar.

APPENDIX A

Notation and glossary

Symbols used in the paper and in this book

Symbol	Meaning	Base model	Chapter
x	one token's vector (one row)	512 numbers	A1
X	matrix of token vectors, one row per position	(n, 512)	A2
n	number of tokens in a sequence	varies	A2
A^T	transpose: rows become columns	—	A2
softmax	turns scores into positive weights that sum to 1	—	A3
d_model	width of every token vector between sublayers	512	C2
W_Q, W_K, W_V	learned projections that make queries, keys and values	(512, 64) per head	D3
Q, K, V	queries, keys and values: XW_Q, XW_K, XW_V	(n, 64) per head	D3
d_k, d_v	width of each query/key vector and each value vector	64	D4
QK^T	table of every query–key dot product	(n, n)	D4
$\sqrt{d_k}$	scale factor applied to the scores	8	D4
h	number of attention heads	8	D5
W_O	output projection after the heads are concatenated	(512, 512)	D5
−∞ (mask)	score given to a forbidden position before softmax	—	D6
PE(pos, i)	positional encoding value for position pos, dimension i	—	D7
d_ff	inner width of the feed-forward network	2048	E1
N	number of blocks in each stack	6	E2
P_drop	dropout rate	0.1	B4
ε_ls	label-smoothing amount	0.1	E6
warmup_steps	steps over which the learning rate ramps up	4000	E6

Terms, A to Z

Activation function A nonlinear function applied after a weighted sum, for example ReLU. B1

Attention A weighted average of value vectors, where the weights come from query–key match scores. D1

Attention weights The softmax outputs: how much each position contributes to one query's mixture. Each row sums to 1. D2

Autoregressive Generating one token at a time, with each new token fed back in as input. E7

Backpropagation	Using the chain rule to compute every parameter's gradient efficiently, from the loss backward. B3
Batch	A group of examples processed together; the gradient is averaged over the batch. B4
Beam search	A decoding method that keeps the k most probable partial outputs at each step. E7
BPE	Byte-pair encoding: builds a subword vocabulary by repeatedly merging the most frequent adjacent pair. C1
Causal mask	Stops a position from attending to later positions. D6
Chain rule	The rate of change along a chain of functions is the product of each link's rate. A4
Cross-attention	Queries come from the decoder; keys and values come from the encoder output. E3
Cross-entropy	The loss $-\ln(p)$, where p is the probability the model gave the correct answer. B2
Decoder	The stack that generates the target, using masked self-attention and cross-attention. E3
Dot product	Multiply matching entries and add. A large value means the vectors are aligned. A1
Dropout	During training, randomly zeroing some activations to reduce overfitting. B4
Embedding	A learned vector stored for each token ID. C2
Encoder	The stack that turns the source sentence into contextual vectors. E2
Feed-forward network	Two linear layers with a ReLU between them, applied to each position separately. E1
Gradient	The vector of derivatives of the loss with respect to every parameter. A4
Head	One independent attention computation with its own projections. D5
Hidden state	An intermediate vector for a position, inside an RNN or a Transformer layer. C3
Inference	Using a trained model. The parameters stay fixed. B3
Label smoothing	Training against a slightly softened target distribution instead of a hard one-hot. E6
Layer normalization	Recentering and rescaling each token vector's features, then applying a learned scale and shift. E1
Learning rate	How far each update moves the parameters. B3
Logits	Raw scores before softmax. B2
Padding	Filler tokens that make sequences in a batch the same length. They are masked out. D6
Positional encoding	A vector added to each embedding so the model can tell positions apart. D7
Query, key, value	Three learned views of each token: what it seeks, what it offers for matching, and what it hands over. D3
Residual connection	Adding a sublayer's input to its output: $x + F(x)$. E1
RNN	A recurrent network that processes tokens one step at a time, carrying a hidden state. C3
Self-attention	Attention where Q, K and V all come from the same sequence. D3
Teacher forcing	During training, feeding the correct previous tokens as decoder inputs. E6
Token	A piece of text (a word, subword, character or byte) with an ID in the vocabulary. C1

Vocabulary	The fixed set of tokens the model knows. About 37,000 in the paper's English–German setup. C1
Warmup	Raising the learning rate gradually at the start of training. E6

APPENDIX B

Solutions to exercises

A1 · Vectors and the dot product as similarity

A1.1 $3\times2 + (-1)\times4 = 6 - 4 = \mathbf{2}$. The result is small and positive, so the angle between them is a little less than 90°. Their cosine similarity is $2 \div (\sqrt{10} \times \sqrt{20}) \approx 0.14$.

A1.2 $a \cdot a = |a|^2$. The tea vector is longer ($\sqrt{2}$ versus 1), so its self-match is 2 rather than 1. The dot product mixes direction with length, while cosine similarity keeps only direction.

A2 · Matrices, shapes and transpose

A2.1 XW is **(5, 3)**: five tokens, each now with three features. W^T is **(3, 8)**.

A2.2 **(10, 10).** Entry (i, j) is the dot product of row i with row j: one score for every pair of the 10 positions.

A2.3 The inner dimensions (2 and 3) don't match, so the rows and columns have different lengths and their dot products are undefined. Transposing the second matrix gives (3, 2) @ (2, 3) → (3, 3).

A3 · Exponentials, logarithms and softmax

A3.1 [0.5, 0.5], because equal scores give equal weights. Then exp gives [1, 3], the total is 4, and the weights are **[0.25, 0.75]**.

A3.2 $\exp(s_i - m) = \exp(s_i) \times \exp(-m)$. Every term, including the total, is multiplied by the same factor $\exp(-m)$, so the factor cancels in the division. The point is to keep the numbers small enough that exp doesn't overflow.

A4 · Derivatives and the chain rule

A4.1 $f'(w) = 6w + 2$, so $f'(1) = \mathbf{8}$.

A4.2 $2(6 - 3)(3) = \mathbf{18}$. The slope is positive, so decrease w. That makes sense: the prediction 6 overshoots the target 3.

A4.3 Each link scales a small change. If u moves 3 units per unit of w, and L moves 2 units per unit of u, then a nudge in w is scaled by 3 and then by 2, so L moves 6 units per unit of w. Scalings compound by multiplying.

A5 · Mean, variance and normalization

A5.1 Both give **[−1, 1]**. Mean 2, standard deviation 1 for the first; mean 20, standard deviation 10 for the second. Normalization removes scale and shift.

A5.2 The variance is 256, so the typical size is $\sqrt{256} = 16$. Dividing by $\sqrt{256} = 16$ restores a typical size of about 1.

B1 · Neurons and layers, starting from familiar ML

B1.1 z = 0.5 × 6 − 2 = 1, h = ReLU(1) = 1, and the prediction is 3 × 1 + 1 = **4**. We changed the input, not the learned rule.

B1.2 W is **(512, 2048)**, which is 1,048,576 weights, plus 2,048 biases: 1,050,624 parameters.

B1.3 Each layer is a matrix multiply plus a bias. A product of matrices is just another matrix, so the ten layers collapse into one XW + b. Only a nonlinearity between the layers prevents the collapse.

B2 · Measuring error: squared error and cross-entropy

B2.1 −ln 0.8 ≈ **0.223**. −ln 0.2 ≈ **1.609**.

B2.2 A right/wrong count is flat almost everywhere: small parameter changes don't change it, so its gradient is zero. Cross-entropy changes smoothly with the probability, which gives a useful gradient even when the answer is already right but not yet confident.

B3 · Gradient descent and backpropagation

B3.1 Gradient = 2(3.2 − 4)(2) = −3.2, so w = 1.6 + 0.32 = **1.92**. The prediction is now 3.84, closing in on 4.

B3.2 Each step multiplies the error (w − 2) by (1 − 8 × lr). At lr = 0.25 that factor is −1, so the error flips sign but never shrinks: w alternates between 0 and 4. Below 0.25 the error shrinks, and above it the error grows.

B4 · Training in practice: batches, Adam and overfitting

B4.1 [5, 5, 0, 5] × 1/0.8 = **[6.25, 6.25, 0, 6.25]**.

B4.2 Overfitting: the model is getting better on data it has seen and worse on data it hasn't. Possible responses: stop early at the best validation checkpoint, add regularization (more dropout, label smoothing, weight decay), or get more data.

B5 · Checkpoint B: neural networks

B5.1 The prediction is 3 and the loss is $(3 - 6)^2 = 9$. The gradient is 2(3 − 6)(3) = −18. New w = 1 + 0.05 × 18 = **1.9**, the new prediction is 5.7 and the new loss is 0.09.

B5.2 A batch goes through the network to produce predictions (the forward pass). A loss measures how wrong they are. Backprop applies the chain rule to get the gradient of the loss with respect to every parameter. The optimizer moves each parameter a small step against its gradient. Repeat.

C1 · Tokenization: from text to IDs

C1.1 *newer*: n e w e r. No merge applies (there's no "e s", "l o" or "lo w" in it), so it stays **n · e · w · e · r**. *slow*: s l o w → s lo w → **s · low**.

C1.2 IDs are arbitrary labels. Treating them as quantities would imply that ID 2 is "between" IDs 1 and 3, or "twice" ID 1. That ordering is meaningless, so we look up a learned vector for each ID instead.

C2 · Embeddings: from IDs to vectors

C2.1 [[1,1],[1,1],[1,0]], with shape **(3, 2)**.

C2.2 Yes, it looks up the same row. No, the final vectors can differ, because the two occurrences sit at different positions (D7) and see different context through attention.

C3 · RNNs and the seq2seq bottleneck

C3.1 It can process many *sentences* at once, but within each sentence the positions must still be computed one after another, because each state needs the previous one. Long sentences mean long chains of dependent steps.

C3.2 All three. More must fit into the fixed vector c, the paths between distant words get longer, and there are twice as many sequential steps.

C4 · Attention before Transformers

C4.1 2014: the decoder looks at all encoder states through a weighted average, which removes the one-vector bottleneck. 2017: self-attention inside each sentence replaces recurrence, which removes the long paths and the sequential computation.

C4.2 0.1[1,0] + 0.2[0,1] + 0.7[2,2] = [0.1, 0] + [0, 0.2] + [1.4, 1.4] = **[1.5, 1.6]**.

D1 · Attention as a weighted average

D1.1 0.75[4,0] + 0.25[0,8] = [3, 0] + [0, 2] = **[3, 2]**. The width stays at 2.

D1.2 The vectors v_1, v_2 and v_3 play the role of the encoder states, and the output [1.4, 1.6] is the context vector. It is the same operation. Self-attention simply applies it within one sentence, for every position at once.

D2 · From scores to weights: one query by hand

D2.1 Scores [1, 0, 1], exp [2.718, 1, 2.718], total 6.437, weights [0.422, 0.155, 0.422]. Output: 0.422[1,0] + 0.155[0,1] + 0.422[1,1] = **[0.845, 0.578]**.

D2.2 tea·tea = 2, because tea is a longer vector, while like·like = 1. The larger score gets a larger softmax share. This is the length effect from A1 showing up inside attention.

D3 · Why queries, keys and values are separate

D3.1 Q is **(7, 64)**, and W_Q holds 512 × 64 = **32,768** numbers. It has the same shape for any sentence length.

D3.2 Q, K and V all come from the same sequence X, the sequence attending to itself. "Self" doesn't mean a position looks only at itself: each one attends over every allowed position.

D3.3 Q and V change, because they are computed from the input. W_Q is a learned parameter and stays fixed after training.

D4 · The matrix form and the $\sqrt{d_k}$ scale

D4.1 exp ≈ [2.028, 2.028, 4.113], total ≈ 8.169, weights ≈ [0.248, 0.248, 0.503]. Output ≈ 0.248[2,0] + 0.248[0,2] + 0.503[2,2] = **[1.503, 1.503]**.

D4.2 Each row is one query's scores over all keys, and that query needs weights summing to 1 to form its own mixture. Normalizing columns would mix up different queries' decisions.

D5 · Multi-head attention

D5.1 **(6, 12)**. The six positions remain, each now with 12 features.

D5.2 1024 / 16 = **64**, the same per-head width as the base model.

D5.3 One head produces a single weight distribution per query, so it can only form one mixture. Eight heads produce eight independent distributions per query, so one position can gather different information from different places at once, for about the same cost.

D6 · Masks: padding and causal

D6.1 Only START and I are allowed: softmax([0, 0.707]) ≈ **[0.330, 0.670, 0, 0]**.

D6.2 "like" is part of the known prefix. The next token, "tea", is the label for that position. Letting the position see the later input "tea" would reveal the answer it is being trained to predict.

D7 · Positional encoding

D7.1 **No.** It adds matching coordinates, so the width stays at d_model. Concatenation would widen the vector; addition doesn't.

D7.2 No. Each word's output would be the same in both sentences (just reordered), because self-attention without position information is permutation-equivariant.

D8 · Checkpoint D: attention from scratch

D8.1 Scores A·[A, B, C] = [1, 0, 2]. Scaled: [0.707, 0, 1.414]. exp ≈ [2.028, 1, 4.113], total 7.141, weights ≈ [0.284, 0.140, 0.576]. Output = 0.284[1,0] + 0.140[0,1] + 0.576[2,0] ≈ **[1.436, 0.140]**.

D8.2 B may see A and B only. The scores B·A = 0 and B·B = 1 scale to [0, 0.707], and softmax gives **[0.330, 0.670, 0]**.

D8.3 Each token is projected three ways: a query (what it seeks), a key (what it offers) and a value (what it hands over). Every query is dotted with every key, giving an n × n score table, scaled by $\sqrt{d_k}$ so softmax isn't too sharp. Softmax on each row gives weights summing to 1, and each token's output is the weighted sum of the values. Several heads do this in parallel with different projections, and their outputs are concatenated and projected back. Masks set forbidden scores to −∞: padding everywhere, and future positions in the decoder.

E1 · Feed-forward, residual connections and layer normalization

E1.1 [2, 4]: mean 3, standard deviation 1, giving **[−1, 1]**. [0, 0, 0, 4]: mean 1, variance (1 + 1 + 1 + 9)/4 = 3, standard deviation ≈ 1.732, giving **[−0.577, −0.577, −0.577, 1.732]**.

E1.2 512 × 2048 + 2048 + 2048 × 512 + 512 = **2,099,712**.

E1.3 **Attention.** The FFN treats each token independently. With only FFNs, no token could ever learn anything about its neighbours, so every word would be processed out of context.

E2 · The encoder block and stack

E2.1 1,050,624 + 2,099,712 + 2,048 = **3,152,384**, about 3.15M. Six blocks make about 18.9M.

E2.2 Self-attention → Add & Norm → FFN → Add & Norm, with a residual path from each sublayer's input to the Add & Norm right after it, and input and output both (n, 512).

E3 · The decoder: masked self-attention and cross-attention

E3.1 Cross-attention: **(9, 12)**. Self-attention: **(9, 9)**, lower-triangular after masking.

E3.2 Encoder self-attention mixes source positions with each other, with no causal mask. Decoder masked self-attention mixes the available target positions while hiding future ones. Cross-attention uses target-side queries to gather information from the encoder's keys and values.

E3.3 The causal mask hides *target* tokens that haven't been generated yet. The source sentence is fully given before decoding starts, so seeing all of it reveals nothing about the answer.

E4 · The output layer: from vectors to next-token probabilities

E4.1 Logits: END = 2.2, I = −1, like = −1.2, tea = −2.2, coffee = −1.96. **END** wins: the model would stop generating here.

E4.2 Each logit is $h \cdot e_j$. If two embeddings are close, their dot products with any h are close, so their logits and probabilities are close.

E5 · One sentence through the whole model: a shape trace

E5.1 Encoder output **(32, 20, 512)**. Decoder self-attention weights **(32, 8, 22, 22)**. Logits **(32, 22, 37000)**.

E5.2 Vocabulary size: the embedding table and the logits. Length squared: the attention weight tables, (n, n) and (target, source).

E6 · How the Transformer is trained

E6.1 Step 1,000 (still warming up): 0.0442 × 1000 × 4000^(−1.5) ≈ **0.000175**. Step 16,000 (decaying): 0.0442 × 16000^(−0.5) ≈ **0.000349**.

E6.2 Every token gets 0.1/10 = 0.01, and the correct one also gets 0.9: index 3 = **0.91**, all others **0.01**. The total is 1.

E6.3 In training, the correct prefix for every position is already known, so all positions can be computed at once, and the mask stops each one from seeing its own answer. In generation, the prefix is the model's own output, which has to be produced one token at a time.

E7 · How it generates: greedy decoding and beam search

E7.1 "The dog" (0.20). Beam search with k = 1 **is** greedy decoding.

E7.2 The source never changes during generation, so its memory is computed once and reused. The decoder's input grows by one token each step, and its next prediction depends on that new token.

E8 · The whole architecture, and how to read the paper

E8.1 Embedding + PE (3, 512) and (4, 512); encoder self-attention with (3, 3) weights per head; FFN with a (3, 2048) inside; memory (3, 512); masked decoder self-attention (4, 4); cross-attention (4, 3); FFN; linear → (4, 37000); softmax. Residual and Add & Norm around every sublayer, and six blocks per stack.

E8.2 It doesn't mean the model has only attention (there are FFNs, embeddings, norms and residuals). It doesn't mean the paper invented attention (it dates from 2014). It doesn't mean output is produced all at once (generation is still token by token).

G1 · Encoder-only, decoder-only and encoder–decoder

G1.1 Its attention is bidirectional, so every position can already see the next token. Predicting it would be trivial copying. Masking tokens out of the input creates a real prediction task.

G1.2 The source is simply part of the same sequence. Masked self-attention lets each output token attend back to the source tokens earlier in the sequence, so self-attention does the job cross-attention did.

G2 · What modern models changed, and what they kept

G2.1 Because of the causal mask: position i only attends to positions ≤ i, so nothing that arrives later can influence its hidden states, and so it can't influence its keys and values either. They can be computed once and reused.

G2.2 SwiGLU → the FFN's activation (E1). RoPE → positional encoding (D7). GQA → multi-head attention's keys and values (D5). RMSNorm → layer normalization (E1).

G3 · Pretraining, fine-tuning and instruction tuning

G3.1 Your message is split into tokens, and each token becomes a list of numbers. Many layers of attention let every token gather information from the others, so each one comes to reflect its context. At the end, the model gives a probability to every possible next token. It picks one, adds it to the text, and repeats, one token at a time, until it decides to stop. All of this uses fixed numbers learned in training. Answering a question doesn't change them.

G3.2 Attention (D1–D5). Each output depends on the whole input through attention weights computed on the fly. Different prompts produce different mixtures, so the same fixed parameters behave differently depending on the examples in context.

APPENDIX C

Answers to self-assessment and review questions

0.2 · Readiness check: where should you start?

Q1 **(a) 11**. **11.** Multiply matching entries and add them: 2×4 + 3×1. *If you missed this, read A1 Dot product as similarity.*

Q2 **(c) [1, 1] and [2, 2]**. **[1,1] and [2,2]** point the same way, so their cosine similarity is 1. The first pair are opposite and the second pair are perpendicular. *If you missed this, read A1 Dot product as similarity.*

Q3 **(b) (5, 3)**. **(5, 3).** The inner dimensions (8 and 8) must match, and the outer ones remain. *If you missed this, read A2 Matrices, shapes and transpose.*

Q4 **(a) [0.67, 0.24, 0.09]**. **[0.67, 0.24, 0.09].** Exponentiate each score (7.39, 2.72, 1) and divide each by their total (11.11). *If you missed this, read A3 Exponentials, logs and softmax.*

Q5 **(b) 6**. **6.** The derivative of w^2 is 2w, so at w = 3 the slope is 6. *If you missed this, read A4 Derivatives and the chain rule.*

Q6 **(a) 1**. **1.** The mean is 3, both deviations are ±1, and the average squared deviation is 1. *If you missed this, read A5 Mean, variance and normalization.*

Q7 **(b) Without it, stacked layers collapse into one linear function**. **Without it, layers collapse.** h = 2x + 1 followed by y = 3h + 4 is just y = 6x + 7. *If you missed this, read B1 Neurons and layers.*

Q8 **(a) lower for 0.9**. **Lower for 0.9.** The loss is −ln(p): −ln(0.9) ≈ 0.11, while −ln(0.1) ≈ 2.30. *If you missed this, read B2 Loss functions.*

Q9 **(b) Increase w**. **Increase w.** The update is w − learning_rate × (−16), which adds a positive amount. *If you missed this, read B3 Gradient descent and backprop.*

Q10 **(a) Looks up a learned vector for each token ID**. **A lookup.** Row i of a learned table is the vector for token ID i. *If you missed this, read C2 Embeddings.*

Q11 **(c) Each step needs the previous step's hidden state**. **The sequential dependency.** Step t cannot start until step t − 1 has finished. *If you missed this, read C3 RNNs and the bottleneck.*

B5 · Checkpoint B: neural networks

Q1 **(c) (512, 2048)**. **(512, 2048).** (7, 512) @ (512, 2048) → (7, 2048). The token count never appears in W.

Q2 **(b) −ln 0.5 ≈ 0.693**. exp gives [1, 1, 2], the total is 4, so p_correct = 2/4 = 0.5. The loss is −ln 0.5 ≈ **0.693**.

Q3 **(a) It computes gradients by applying the chain rule from the loss backward**. **It computes gradients.** The optimizer then updates the weights. Neither happens during ordinary inference.

Q4 **(d) overfitting**. **Overfitting:** the model is memorizing the training data.

Q5 **(b) make the weights more concentrated on the top score**. **More concentrated.** Scaling spreads the scores apart. Only *adding* a constant leaves softmax unchanged.

Q6 **(c) 10**. **10.** The variance is 100, so the standard deviation is $\sqrt{100} = 10$. The mean is 0, but a typical value is about ±10.

C5 · Checkpoint C: text and sequences

Q1 **(b) (32000, 512)**. **(32000, 512)**: one row per token.

Q2 **(c) Any word can fall back to smaller pieces, down to characters or bytes**. **Fallback to smaller pieces:** unseen words split into known subwords.

Q3 **(a) Squeezing the whole source into one fixed vector**. **The bottleneck.** Sequential computation remained until the Transformer.

Q4 **(d) It removed recurrence and relies on self-attention to mix positions**. The other three are common misreadings. Generation is still one token at a time (E7). What runs in parallel is training on known targets.

Q5 **(b) [0, 1]**. **[0, 1]**: row 1 of E, the vector for "like".

D8 · Checkpoint D: attention from scratch

Q1 **(c) (10, 10)**. **(10, 10):** one row per query and one column per key.

Q2 **(a) Dot products grow with width; scaling keeps softmax from saturating, where gradients vanish**. Softmax already ensures a sum of 1. The scale controls how *sharp* the weights are.

Q3 **(d) setting future scores to −∞ before softmax**. **−∞ before softmax** gives zero weight and keeps each row summing to 1.

Q4 **(b) (n, 512)**. Concatenation gives (n, 512), and W_O (512 × 512) keeps it at (n, 512).

Q5 **(c) shuffle the outputs in the same way, each token keeping the same vector**. Self-attention is **permutation-equivariant**. That is why position must be added.

Q6 **(a) W_Q**. X and Q depend on the input sentence. W_Q is the parameter.

APPENDIX D

References

Papers and resources recommended in the chapters, in order of first mention.

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). Attention Is All You Need. https://arxiv.org/abs/1706.03762
2. 3Blue1Brown, Essence of Linear Algebra. https://www.3blue1brown.com/topics/linear-algebra
3. 3Blue1Brown, Essence of Calculus. https://www.3blue1brown.com/topics/calculus
4. 3Blue1Brown, Neural Networks, chapter 1. https://www.3blue1brown.com/topics/neural-networks
5. Kingma & Ba (2014), Adam. https://arxiv.org/abs/1412.6980
6. Srivastava et al. (2014), Dropout. https://jmlr.org/papers/v15/srivastava14a.html
7. Sennrich et al. (2016), BPE for neural machine translation. https://arxiv.org/abs/1508.07909
8. Sutskever et al. (2014), Sequence to Sequence Learning. https://arxiv.org/abs/1409.3215
9. Bahdanau, Cho & Bengio (2014), Neural Machine Translation by Jointly Learning to Align and Translate. https://arxiv.org/abs/1409.0473
10. Jay Alammar, The Illustrated Transformer. https://jalammar.github.io/illustrated-transformer/
11. He et al. (2015), Deep Residual Learning. https://arxiv.org/abs/1512.03385
12. Ba et al. (2016), Layer Normalization. https://arxiv.org/abs/1607.06450
13. Szegedy et al. (2015). https://arxiv.org/abs/1512.00567
14. Devlin et al. (2018), BERT. https://arxiv.org/abs/1810.04805
15. Brown et al. (2020), GPT-3: Language Models are Few-Shot Learners. https://arxiv.org/abs/2005.14165
16. Raffel et al. (2019), T5. https://arxiv.org/abs/1910.10683
17. Xiong et al. (2020), On Layer Normalization in the Transformer Architecture. https://arxiv.org/abs/2002.04745
18. Su et al. (2021), RoFormer: rotary position embeddings. https://arxiv.org/abs/2104.09864
19. Dao et al. (2022), FlashAttention. https://arxiv.org/abs/2205.14135
20. Ainslie et al. (2023), GQA. https://arxiv.org/abs/2305.13245
21. Shazeer (2020), GLU Variants Improve Transformer. https://arxiv.org/abs/2002.05202
22. Ouyang et al. (2022), Training language models to follow instructions with human feedback. https://arxiv.org/abs/2203.02155